



UNIVERSIDAD
DE GRANADA



Técnicas de los Sistemas Inteligentes

Grado en Informática

Curso 2025-26. Práctica 1

Búsqueda en tiempo real: RTA* y LRTA*(k)

Pablo Mesejo Santiago y David Criado Ramón

**Departamento de Ciencias de la
Computación e Inteligencia Artificial**

<http://decsai.ugr.es>

	Algoritmos	Características
Búsqueda no informada	BFS, DFS, Dijkstra, ...	Al ser una búsqueda no informada (sin información heurística), son más ineficientes y/o no garantizan que se encuentre el camino óptimo.
Búsqueda heurística (offline)	A*, IDA*, ...	Siempre encuentran el camino óptimo (bajo ciertas condiciones) y lo hacen de forma más eficiente que la búsqueda no informada (debido al uso de información heurística).
Búsqueda heurística online (incremental o en tiempo real)	D*, RTA*, LRTA*, ...	Aunque <u>no garantizan el camino óptimo</u> , permiten enfrentarse a problemas más complejos (entornos desconocidos y/o dinámicos, movimientos rápidos, etc).

	Algoritmos	¿Funciona en entornos desconocidos?	¿Es capaz de realizar movimientos rápidos (tiempo corto)?
Búsqueda heurística offline	A*, IDA*, ...	NO	NO
Búsqueda incremental	D*, LPA*, D*Lite, ...	SÍ	NO
Búsqueda en tiempo real	RTA*, LRTA*, LRTA*(k) ...	SÍ	SÍ

Búsqueda offline

BÚSQUEDA

EJECUCIÓN

Búsqueda incremental

BÚSQUEDA

EJECUCIÓN

BÚSQUEDA

EJECUCIÓN

BÚSQUEDA

EJECUCIÓN



Búsqueda en tiempo real

BÚSQUEDA

EJECUCIÓN

BÚSQUEDA

EJECUCIÓN

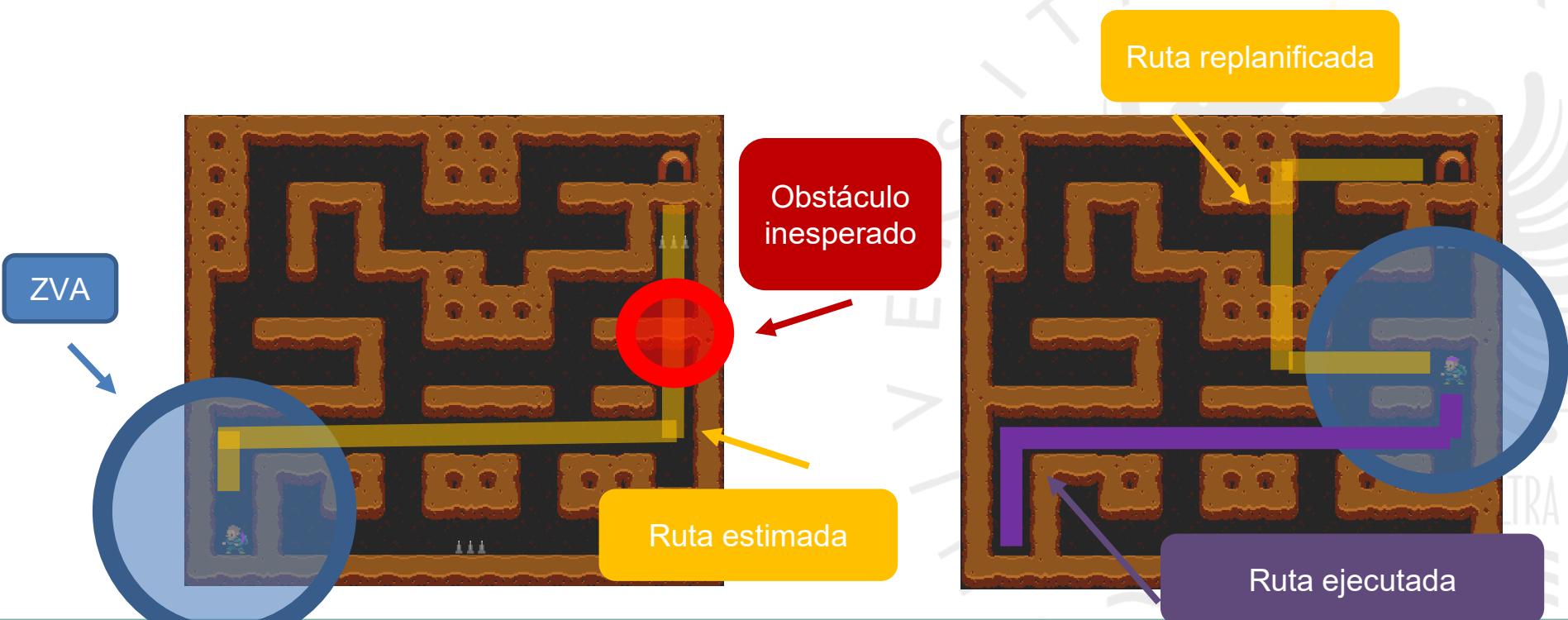


- La búsqueda heurística clásica (offline) requiere **información completa del entorno**.
- Con esta información, primero se realiza la búsqueda de la ruta (normalmente óptima) –proceso más costoso– y posteriormente se ejecuta –proceso más rápido–.
- Pero en algunos contextos, el entorno solo se conoce parcialmente:
 - **Información desconocida**. Ej: alcance de los sensores del robot.
 - **Información dinámica**: Ej: enemigos que se mueven por el entorno.
- En estos casos, es **inviable calcular UN único plan** (como se hace en la búsqueda heurística clásica), dado que éste puede sufrir modificaciones.
- ¡Es necesario **replanificar**!

Búsqueda incremental



- La **búsqueda incremental** aborda el problema de la información incompleta o dinámica.
 - Zona de visibilidad del agente** (ZVA): área abarcada por los sensores.
 - Hipótesis del mundo desconocido**: todo terreno desconocido es navegable.
- Ejemplo: un método incremental basado en A*. Calcular un plan, intentar ejecutar y, en caso de obstáculo, recalcular.



- Problema: ¿cómo **actualizar la información heurística** a medida que el agente se mueve por el mapa?
 - Si cada llamada a A^* es “independiente”, no se actualiza.
 - El agente puede caer en **ciclos infinitos**.

2: Se encuentra un nuevo obstáculo en la ruta replanificada, por lo que se replanifica de nuevo



3: La nueva ruta replanificada coincide con la ruta inicial (que tiene un obstáculo).
PROBLEMA: Se ha entrado en un ciclo infinito y el agente queda atrapado

1: Ruta inicialmente estimada. Al ejecutar, se encuentra un obstáculo así que se replanifica una nueva ruta

- La **búsqueda incremental** aborda este problema.

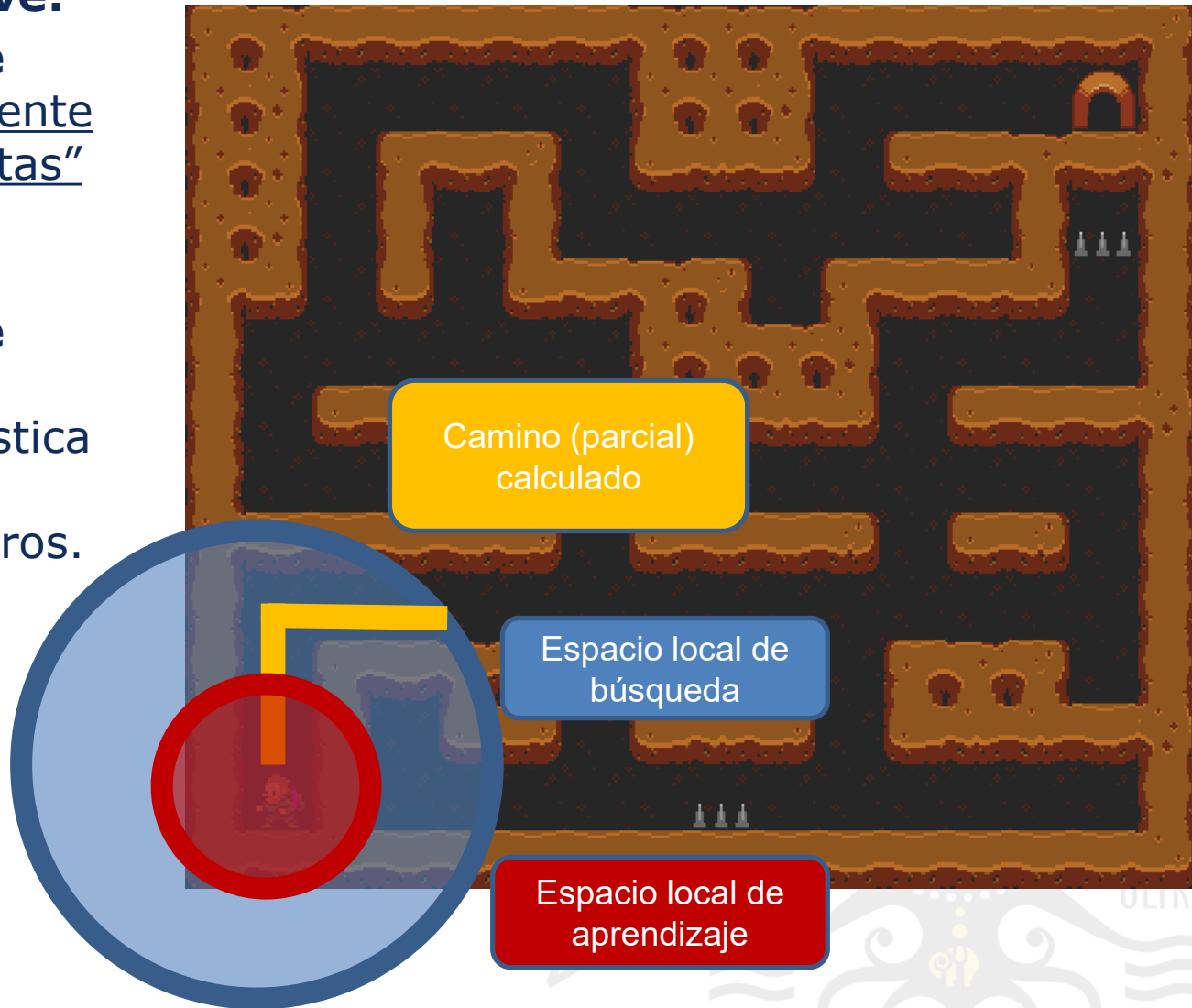
- Algoritmos de **búsqueda incremental**: D*, LPA*, D*Lite,...
- La idea general es mantener, de la forma más eficiente posible, **información actualizada sobre $h(n)$** a medida que **el agente aprende nueva información sobre el mapa en los distintos intentos de ejecutar la ruta.**
- Inconveniente: **en el peor caso**, la búsqueda de un camino es tan **costosa** como en búsqueda offline.
 - Los tiempos de planificación (búsqueda) pueden ser muy altos.
 - Por tanto, **no sirve para realizar movimientos rápidos.**
- Ante la necesidad de movimientos/decisiones rápidas aparece la **búsqueda en tiempo real.**

LPA*: [Koenig, Likhachev, Furcy. 2004. Lifelong Planning A*. *Artificial Intelligence* 155 (1–2): 93-146]
D*: [Stentz. 1994. Optimal and Efficient Path Planning for Partially-Known Environments. *Proc. of ICRA*]
D*Lite: [Koenig, Likhachev. 2005. Fast Replanning for Navigation in Unknown Terrain. *Transactions on Robotics* 21 (3)]

Búsqueda en tiempo real

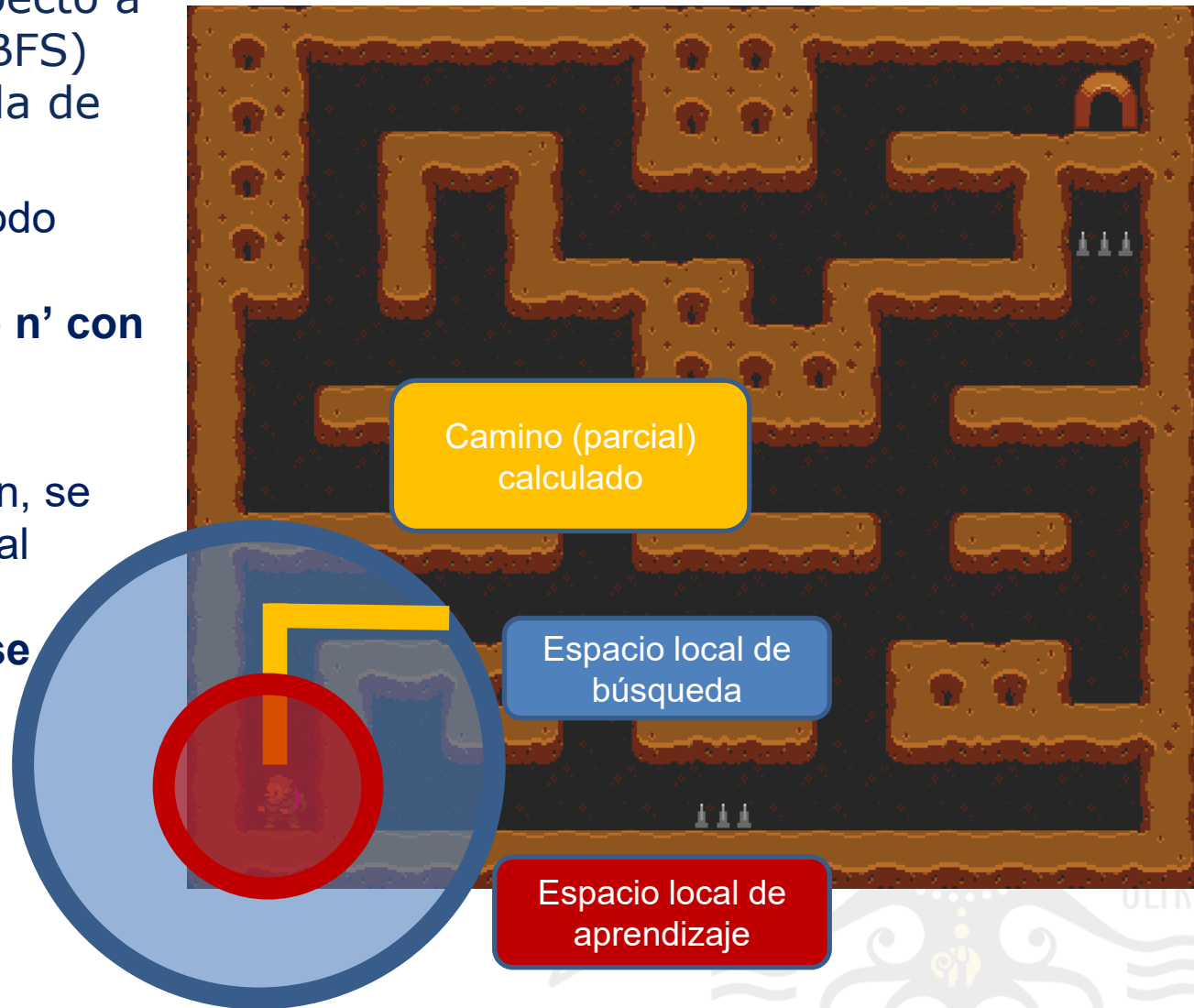
Dos conceptos clave:

- **Espacio local de búsqueda:** el agente calcula rutas "cortas" hasta la mejor frontera.
- **Espacio local de aprendizaje:** se actualiza la heurística de cara a movimientos futuros.



La **diferencia** con respecto a la **búsqueda local** (GBFS) es precisamente la regla de aprendizaje.

- En GBFS, desde un nodo actual n , se realiza un **movimiento al vecino n' con menor $c(n, n') + h(n')$** .
- En los algoritmos que veremos en esta sesión, se realiza un movimiento al vecino n' con menor $c(n, n') + h(n')$, **y luego se actualiza $h(n)$** .



Real-Time A* (RTA*)

- En cada paso, moverse al nodo del espacio local de búsqueda seleccionado por la estrategia de movimiento, y actualizar el espacio local de aprendizaje según la regla de aprendizaje.

- Espacio local de búsqueda: los vecinos del nodo actual x .

- Estrategia de movimiento: mover al mejor vecino z .

$$z = \operatorname{argmin}_{(y \in \text{Sucesores}(x))} \{c(x, y) + h(y)\}$$

- Espacio local de aprendizaje: nodo actual.

- Regla de aprendizaje: solo del nodo actual.

$$h(x) = \max(h(x), 2^{\circ} \min(c(x, y) + h(y)))$$

En cada iteración se comprueba si el nodo expandido es el nodo objetivo

¿De qué forma se relacionan el tamaño de la ruta y el número de nodos expandidos?



RTA*: [Korf. 1990. Real-time Heuristic Search. *Artificial Intelligence* **42**, 189-211]

Real-Time A* (RTA*)

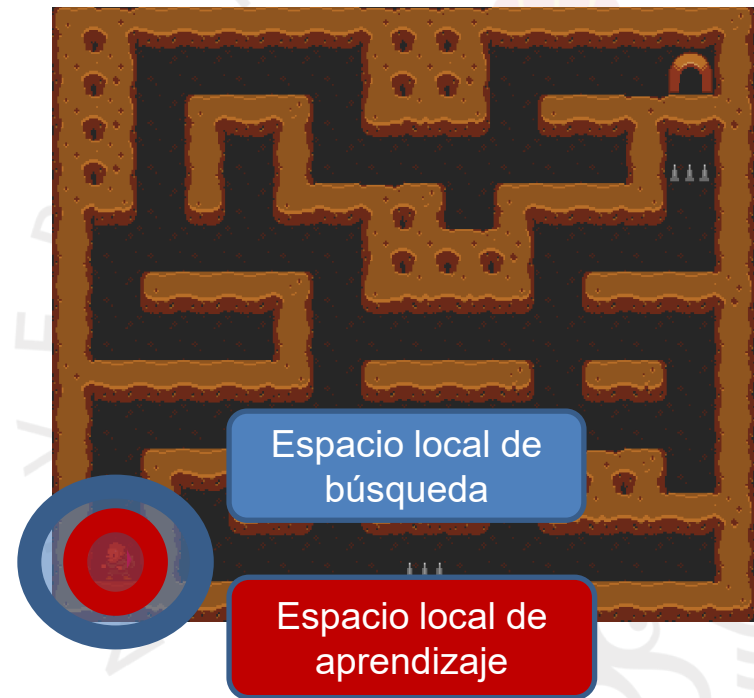
- $actual = nodo_inicial$
- **while True:**
 - **if** $actual == objetivo$:
 - **break** # Se expande el nodo objetivo, fin.
 - # Vecinos del *nodo actual* (espacio local de búsqueda).
 - $S = sucesores(actual)$
 - **foreach** $sucesor$ **in** S :
 - # Calculamos el coste de cada sucesor, y seleccionamos el mejor sucesor en base al coste calculado (estrategia de movimiento).
 - $f(sucesor) = h(sucesor) + distance(actual, sucesor)$
 - $z = argmin(f(y) \mid y \in S)$
 - # Determinamos el segundo mínimo.
 - $segundo_min = Segundo_Mínimo(\{ f(y) \mid y \in S \})$
 - # Actualizamos valor heurístico del nodo actual (regla de aprendizaje).
 - $h(actual) = max(h(actual), segundo_min)$
 - # Nos movemos al mejor vecino.
 - $actual = z$

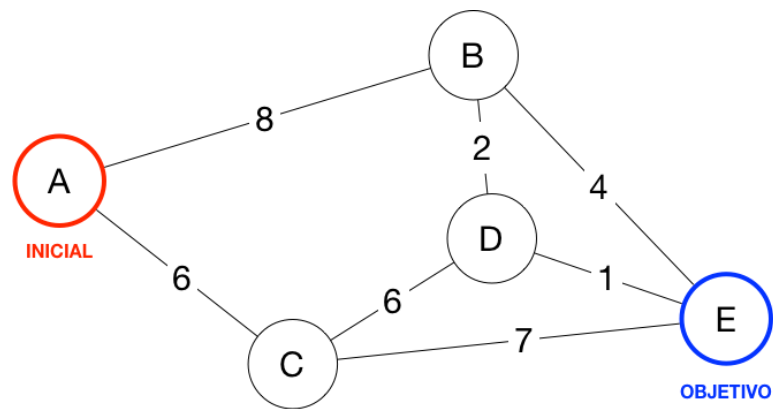
(nodo) actual
es un nodo
expandido!

- **Los valores de $h(n)$** son dinámicos, **se van aprendiendo a lo largo de la ejecución** del algoritmo.
- Cuando se visita un nodo por primera vez, se inicializa su valor con alguna función heurística.
 - En nuestro caso: distancia Manhattan al objetivo.
- Este valor se guarda en una **tabla hash**.
 - Estructura de datos eficiente para acceder y actualizar valores.
- **Cuando un nodo pertenece al espacio local de aprendizaje, se actualiza su $h(n)$.**
 - Simplemente se modifica el valor de este nodo en la tabla hash.

- Espacio local de búsqueda: vecinos del nodo.
- Estrategia de movimiento: mejor vecino.
 - ARRIBA: $c(\text{ACTUAL}, \text{ARRIBA}) + h(\text{ARRIBA}) = 1 + 23 = 24$
 - ABAJO: Muro
 - IZQUIERDA: Muro
 - DERECHA: Muro
- Espacio local de aprendizaje: nodo actual.
- Regla de aprendizaje:
 - $h(\text{ACTUAL}) = 24$

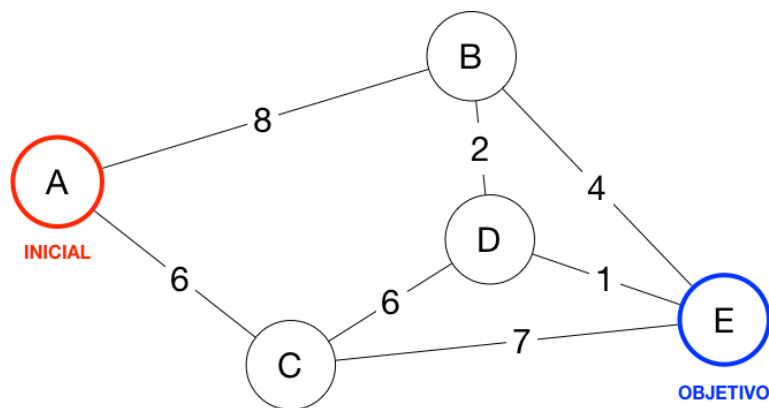
Como no hay segundo mínimo, elegimos el primero





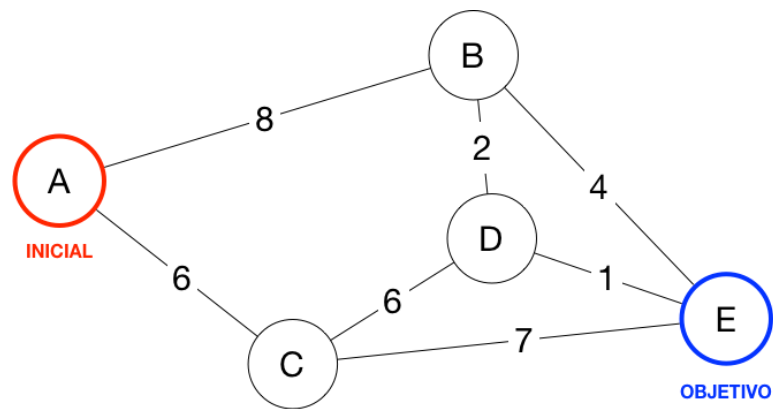
	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Nodo actual	c(x,y)+h(y) en vecinos del nodo actual					Sig. nodo	Nuevo valor h(actual)				
		y=A	y=B	y=C	y=D	y=E		h(A)	h(B)	h(C)	h(D)	h(E)
0								10	4	5	1	0



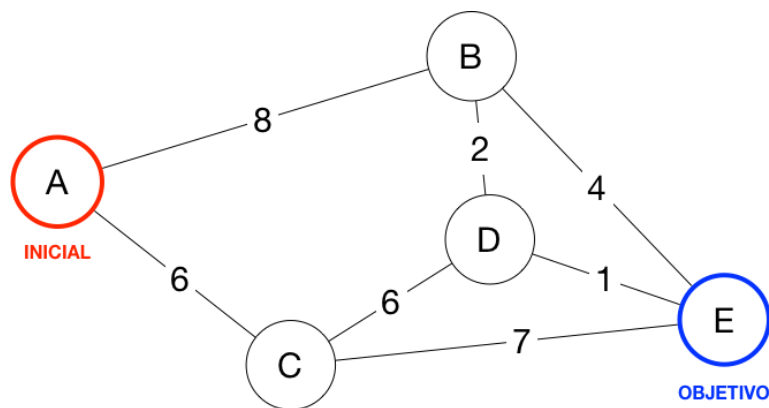
	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Nodo actual	c(x,y)+h(y) en vecinos del nodo actual					Sig. nodo	Nuevo valor h(actual)				
		y=A	y=B	y=C	y=D	y=E		h(A)	h(B)	h(C)	h(D)	h(E)
0								10	4	5	1	0
1	A	-	12	11	-	-	C	12	=	=	=	=



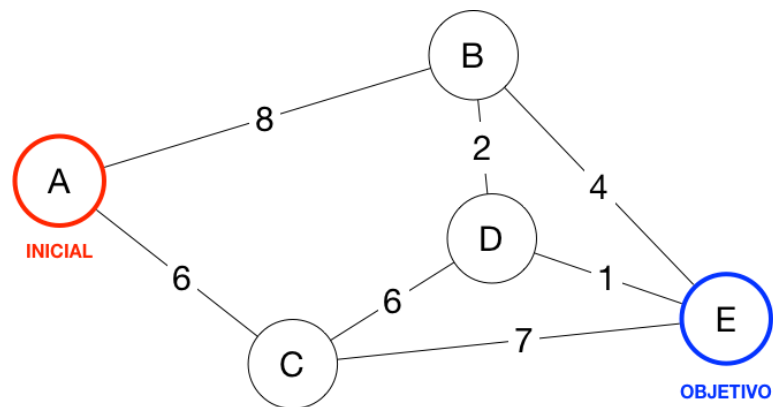
	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Nodo actual	c(x,y)+h(y) en vecinos del nodo actual					Sig. nodo	Nuevo valor h(actual)				
		y=A	y=B	y=C	y=D	y=E		h(A)	h(B)	h(C)	h(D)	h(E)
0								10	4	5	1	0
1	A	-	12	11	-	-	C	12	=	=	=	=
2	C	18	-	-	7	7	D	=	=	7	=	=



	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Nodo actual	c(x,y)+h(y) en vecinos del nodo actual					Sig. nodo	Nuevo valor h(actual)				
		y=A	y=B	y=C	y=D	y=E		h(A)	h(B)	h(C)	h(D)	h(E)
0								10	4	5	1	0
1	A	-	12	11	-	-	C	12	=	=	=	=
2	C	18	-	-	7	7	D	=	=	7	=	=
3	D	-	6	13	-	1	E	=	=	=	6	=



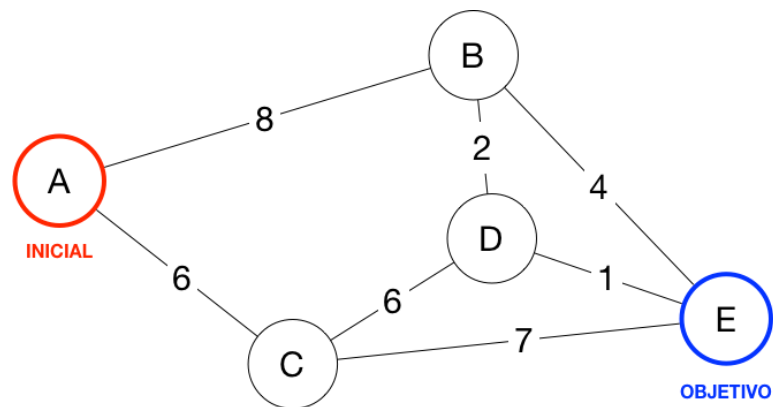
	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Nodo actual	c(x,y)+h(y) en vecinos del nodo actual					Sig. nodo	Nuevo valor h(actual)				
		y=A	y=B	y=C	y=D	y=E		h(A)	h(B)	h(C)	h(D)	h(E)
0								10	4	5	1	0
1	A	-	12	11	-	-	C	12	=	=	=	=
2	C	18	-	-	7	7	D	=	=	7	=	=
3	D	-	6	13	-	1	E	=	=	=	6	=
4	E	Fin. Camino: E – D – C – A						12	4	7	6	0

Learning Real-Time A* (LRTA*)

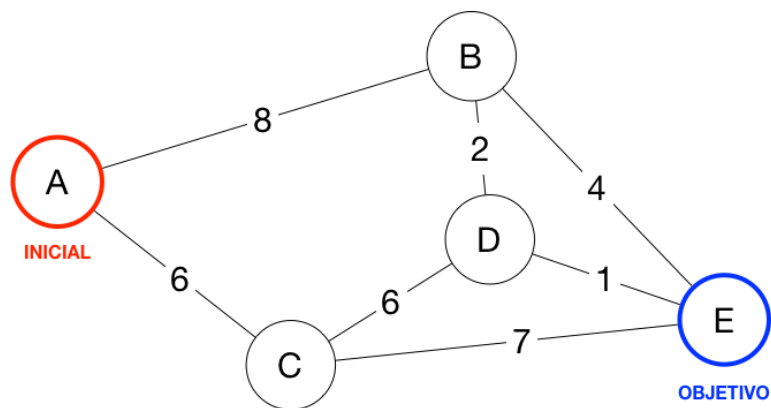
- La única diferencia entre RTA* y LRTA* es la regla de aprendizaje:
 - En RTA* se usa el segundo mínimo.
 - En LRTA* se usa el (primer) mínimo.
- Ambas permiten al agente escapar de mínimos locales en $h(n)$.
 - Zonas prometedoras según la $h(n)$ pero que no llevan a la solución.
- **RTA* puede sobreestimar $h(n)$ por lo que escapa de mínimos locales más rápido.**
 - En una ejecución, es más rápido.
- **LRTA* aproxima mejor $h(n)$ a $h^*(n)$** por lo que es más útil si se requieren múltiples ejecuciones en el mismo mapa.
 - En múltiples ejecuciones, converge más rápido al camino óptimo.
 - De ahí su nombre, pues pretende **aprender** a aproximar mejor $h^*(n)$.

LRTA*: [Korf. 1990. Real-time Heuristic Search. *Artificial Intelligence* **42**, 189-211]



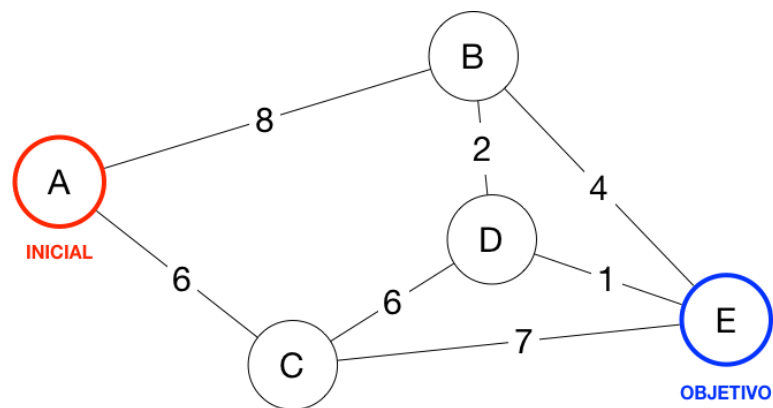
	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Nodo actual	c(x,y)+h(y) en vecinos del nodo actual					Sig. nodo	Nuevo valor h(actual)				
		y=A	y=B	y=C	y=D	y=E		h(A)	h(B)	h(C)	h(D)	h(E)
0								10	4	5	1	0



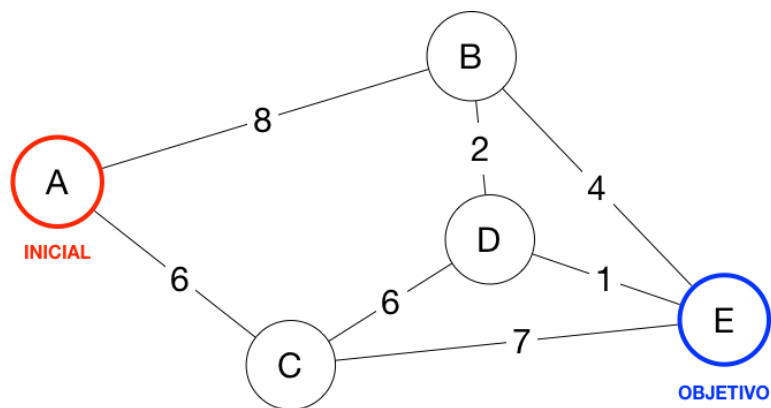
	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Nodo actual	c(x,y)+h(y) en vecinos del nodo actual					Sig. nodo	Nuevo valor h(actual)				
		y=A	y=B	y=C	y=D	y=E		h(A)	h(B)	h(C)	h(D)	h(E)
0								10	4	5	1	0
1	A	-	12	11	-	-	C	11	=	=	=	=



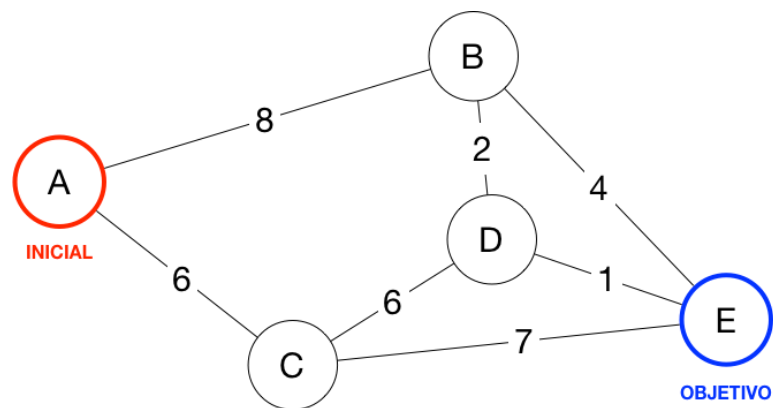
	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Nodo actual	c(x,y)+h(y) en vecinos del nodo actual					Sig. nodo	Nuevo valor h(actual)				
		y=A	y=B	y=C	y=D	y=E		h(A)	h(B)	h(C)	h(D)	h(E)
0								10	4	5	1	0
1	A	-	12	11	-	-	C	11	=	=	=	=
2	C	17	-	-	7	7	D	=	=	7	=	=



	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Nodo actual	c(x,y)+h(y) en vecinos del nodo actual					Sig. nodo	Nuevo valor h(actual)				
		y=A	y=B	y=C	y=D	y=E		h(A)	h(B)	h(C)	h(D)	h(E)
0								10	4	5	1	0
1	A	-	12	11	-	-	C	11	=	=	=	=
2	C	17	-	-	7	7	D	=	=	7	=	=
3	D	-	6	13	-	1	E	=	=	=	1	=

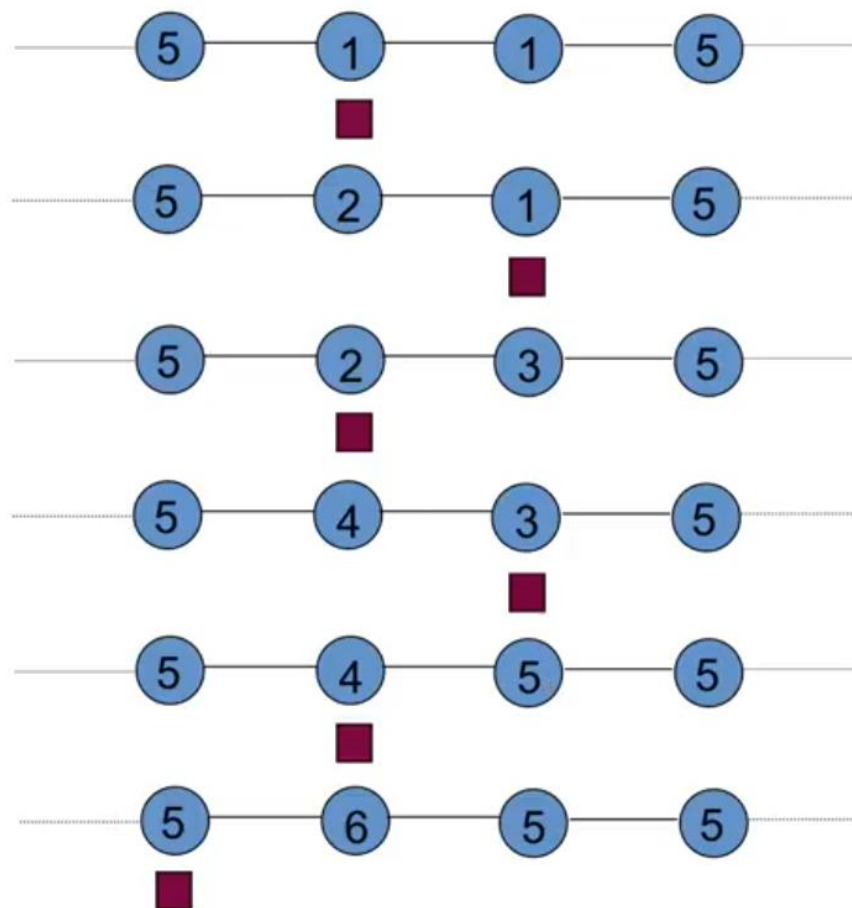


	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Nodo actual	c(x,y)+h(y) en vecinos del nodo actual					Sig. nodo	Nuevo valor h(actual)				
		y=A	y=B	y=C	y=D	y=E		h(A)	h(B)	h(C)	h(D)	h(E)
0								10	4	5	1	0
1	A	-	12	11	-	-	C	11	=	=	=	=
2	C	17	-	-	7	7	D	=	=	7	=	=
3	D	-	6	13	-	1	E	=	=	=	1	=
4	E	Fin. Camino: E – D – C – A						11	4	7	1	0

LRTA*

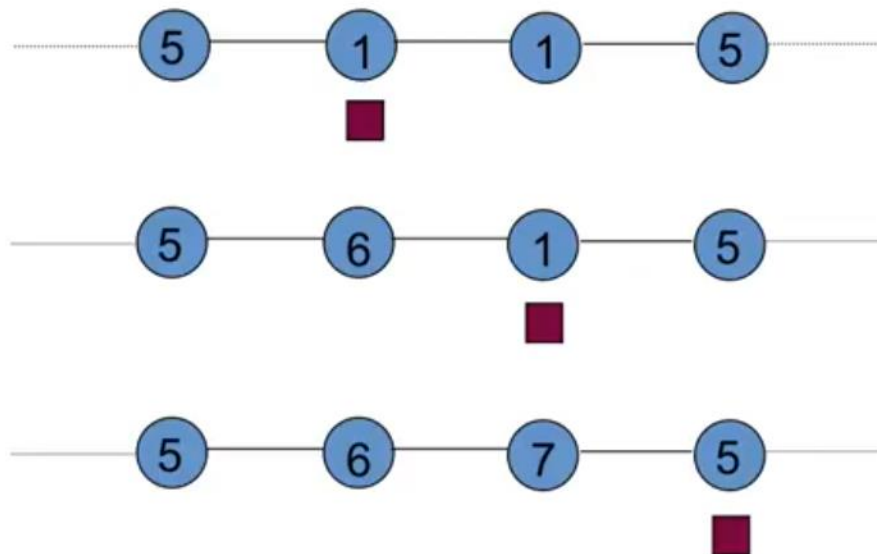
$$h(x) \leftarrow \max\{h(x), \min c(x,y) + h(y)\}$$



Ejemplo extraído de la presentación de Pedro Meseguer sobre Búsqueda Heurística Avanzada (Máster Universitario en IA, AEPIA)

$$h(x) \leftarrow \max\{h(x), 2\text{nd min } c(x,y)+h(y)\}$$

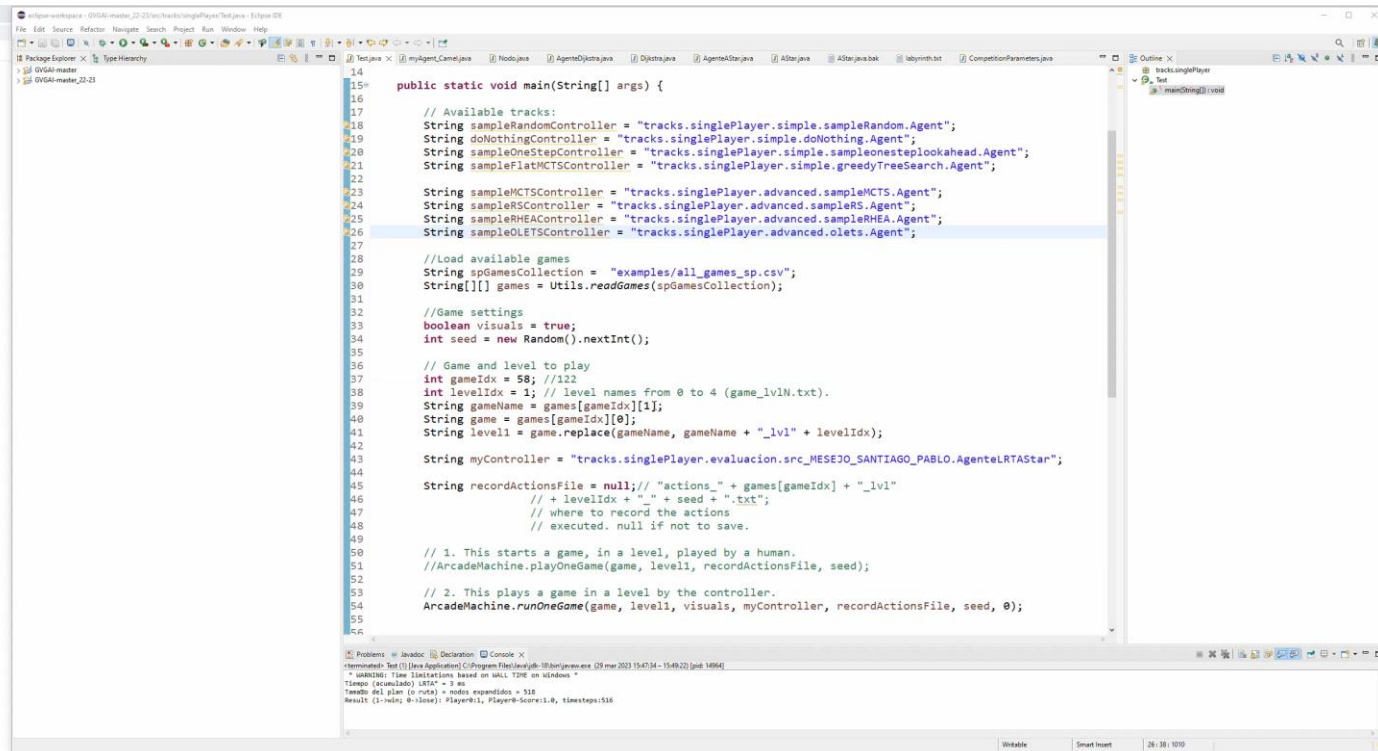
RTA*



Consigue escapar de mínimos locales (depresión heurística) con más rapidez → es más rápido que LRTA*.

Ejemplo extraído de la presentación de Pedro Meseguer sobre Búsqueda Heurística Avanzada (Máster Universitario en IA, AEPIA)

Pero... LRTA* acaba aproximando mejor $h(n)$ a $h^*(n)$ (en múltiples ejecuciones, en donde se parte de la h aprendida en la ejecución anterior).



```

14 public static void main(String[] args) {
15
16     // Available tracks:
17     String sampleRandomController = "tracks.singlePlayer.simple.sampleRandom.Agent";
18     String doNothingController = "tracks.singlePlayer.simple.doNothing.Agent";
19     String sampleOneStepController = "tracks.singlePlayer.simple.sampleOneStepLookahead.Agent";
20     String sampleFlatMCTSController = "tracks.singlePlayer.simple.greedyTreeSearch.Agent";
21
22
23     String sampleMCTSController = "tracks.singlePlayer.advanced.sampleMCTS.Agent";
24     String sampleRHEACController = "tracks.singlePlayer.advanced.sampleRHEA.Agent";
25     String sampleOLETSController = "tracks.singlePlayer.advanced.olets.Agent";
26
27
28     //Load available games
29     String spGamesCollection = "examples/all_games_sp.csv";
30     String[][] games = Utils.readGames(spGamesCollection);
31
32     //Game settings
33     boolean visuals = true;
34     int seed = new Random().nextInt();
35
36     // Game and level to play
37     int gameId = 58; //122
38     int levelId = 1; // level names from 0 to 4 (game_lvlN.txt).
39     String gameName = games[gameId][1];
40     String game = games[gameId][0];
41     String level = game.replace(gameName, gameName + "_lvl" + levelId);
42
43     String myController = "tracks.singlePlayer.evaluacion.src_MESEJO_SANTIAGO_PABLO.AgentLRTAstar";
44
45     String recordActionsFile = null; // "actions_" + games[gameId] + "_lvl"
46     // + levelId + " " + seed + ".txt";
47     // where to record the actions
48     // executed. null if not to save.
49
50     // 1. This starts a game, in a level, played by a human.
51     //ArcadeMachine.playOneGame(game, level, recordActionsFile, seed);
52
53     // 2. This plays a game in a level by the controller.
54     ArcadeMachine.runOneGame(game, level, visuals, myController, recordActionsFile, seed, 0);
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

El primer algoritmo del vídeo es LRTA*: tarda más en alcanzar el destino pero, en sucesivas ejecuciones, aproximaría mejor $h^*(n)$.

El segundo es RTA*: alcanza mucho antes el destino (al sobreestimar la $h(n)$, sale antes de mínimos locales).

Ideas Principales

- **Buscar en un espacio local** en torno al estado actual.
- Como resultado de la búsqueda:
 - **La heurística de algunos estados cambia** (*learning space*).
- **La solución NO es óptima.**
 - Pero puede converger a optimalidad tras ejecuciones repetidas.
 - Se toma como entrada la h final de la resolución anterior.
 - Si h es admisible, LRTA* converge a la solución óptima tras ejecuciones repetidas (RTA* no necesariamente).
- Búsqueda online: **Entrelaza búsqueda y ejecución.**



LRTA*(k)

- Versión parametrizada de LRTA* con mejor rendimiento a costa de un mayor tiempo de procesamiento.
- **Propaga los cambios heurísticos encontrados en hasta k nodos** si el cambio proviene de su soporte (mejor vecino).
- Espacio local de búsqueda: los vecinos del nodo actual x .
- Estrategia de movimiento: mover al mejor vecino z :

$$z = \operatorname{argmin}_{(y \in \text{Sucesores}(x))} \{c(x, y) + h(y)\}$$

- Espacio local de aprendizaje: nodo actual y hasta k nodos sucesores del actual (que ya hayan sido visitados).
- Regla de aprendizaje (nodo actual y sucesores):

$$h(x) = \max(h(x), \min(c(x, y) + h(y)))$$
- Soporte: Para cada estado x se guarda su mejor vecino (sucesor de menor coste; llamado "soporte"). Solo se hace la propagación si esta proviene de su mejor vecino (eficiencia).

+ Diferencia con respecto a LRTA*

LRTA*(k): [[Hernández & Meseguer 2005. International Joint Conference on Artificial Intelligence \(IJCAI\)](#)]

LRTA*(k)

- *actual* = *nodo_inicial*
- **forall** *nodo* **in** *Grafo*:
 - *soporte*(*nodo*) = *null*
- **while** **True**:

- **if** *actual* == *objetivo*:
 - **break** # se expande el nodo objetivo, fin.

(nodo) actual
es un nodo
expandido!

Calculamos la heurística y la propagamos en hasta k nodos.

- **LookaheadUpdateK**(*actual*, *k*)

Seleccionamos el mejor sucesor/vecino.

- $mejorVecino = \operatorname{argmin}_{y \in \text{Sucesores}(actual)} \{c(actual, y) + h(y)\}$

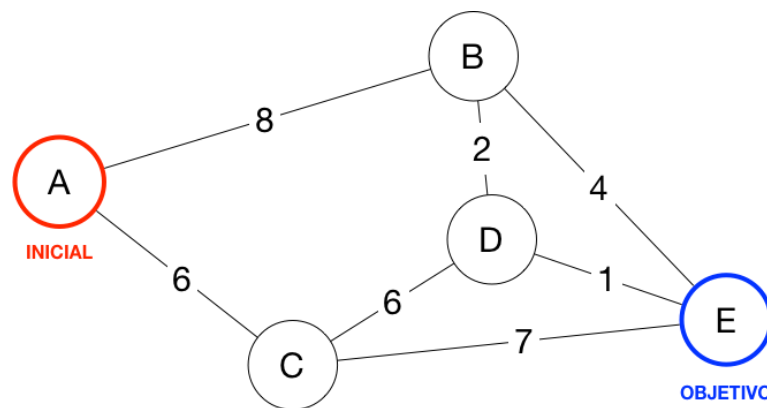
Nos movemos al mejor vecino con la acción correspondiente.

- *actual* = *mejorVecino*

LRTA*(k)

- **PROCEDURE LookaheadUpdateK(actual, k):**
 - Cola = <actual> **# Creamos una cola con el nodo actual.**
 - contador = k-1
 - **while !Cola.empty(): # Mientras que haya elementos en la cola.**
 - $x = \text{Cola.pop}()$ **# Extraemos el primer elemento de la cola.**
 - **# Calculamos el mejor vecino (soporte).**
 - $\text{mejorVecinoX} = \text{argmin}_{(y \in \text{Sucesores}(x))} \{c(x, y) + h(y)\}$
 - $\text{soporte}(x) = \text{mejorVecinoX}$ **# Desempate siguiendo orden de expansión.**
 - **# Actualización mediante la regla de aprendizaje de LRTA*.**
 - $\text{propagar} = \text{False}$
 - **if** $h(x) < c(x, \text{mejorVecinoX}) + h(\text{mejorVecinoX})$:
 - $\text{propagar} = \text{True}$
 - **# Aquí se aumenta en 1 el n° de actualizaciones de h**
 - $h(x) = c(x, \text{mejorVecinoX}) + h(\text{mejorVecinoX})$
 - **if** propagar :
 - **# Intentamos propagar el cambio a los nodos de los que x es su mejor vecino (es decir, el nodo x es su soporte).**
 - **foreach** sucesor **in** $\text{Sucesores}(x)$:
 - **if** $\text{contador} > 0$ **and** $\text{soporte}(\text{sucesor}) == x$:
 - Cola.insert(sucesor)
 - contador=contador-1

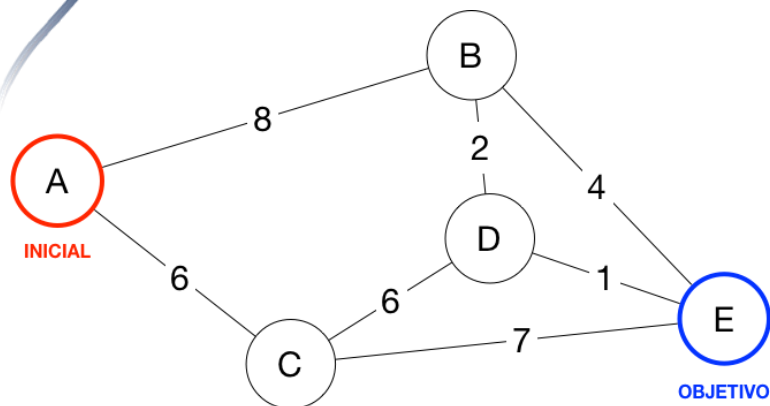
Veamos un ejemplo con LRTA*(k) con $k=2$



	A	B	C	D	E
$h(n)$	10	4	5	1	0

It.	Nodo Actual (Coste)	Camino recorrido	Sig. nodo	Tabla heurística y soporte				
				$h(A)$	$h(B)$	$h(C)$	$h(D)$	$h(E)$
0	-	-	-	10 ($\emptyset$)	4 ($\emptyset$)	5 ($\emptyset$)	1 ($\emptyset$)	0 ($\emptyset$)

Entre paréntesis se marca el soporte del nodo



c(x,y)+h(y) en vecinos del nodo actual				
y=A	y=B	y=C	y=D	y=E
	12	11		

Estando en A

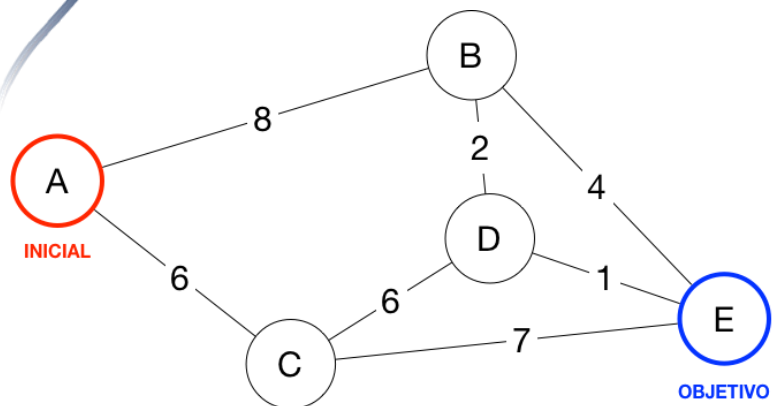
	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Nodo Actual (Coste)	Camino recorrido	Sig. nodo	Tabla heurística y soporte				
				h(A)	h(B)	h(C)	h(D)	h(E)
0	-	-	-	10 (Ø)	4 (Ø)	5 (Ø)	1 (Ø)	0 (Ø)
1	A (0)	A	-	11 (C)	?	?	=	=

$$c(A,C)+h(C)=6+5$$



Hemos actualizado el nodo A. ¿Deberíamos actualizar a sus sucesores (B y C) por la propagación?



c(x,y)+h(y) en vecinos del nodo actual				
y=A	y=B	y=C	y=D	y=E
	12	11		

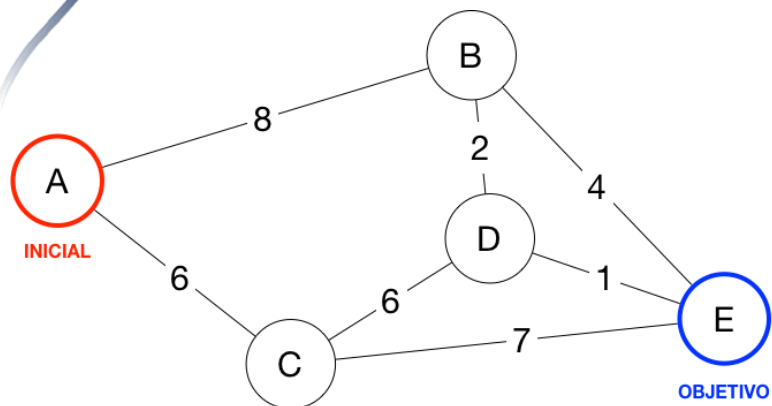
Estando en A

	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Nodo Actual (Coste)	Camino recorrido	Sig. nodo	Tabla heurística y soporte				
				h(A)	h(B)	h(C)	h(D)	h(E)
0	-	-	-	10 (Ø)	4 (Ø)	5 (Ø)	1 (Ø)	0 (Ø)
1	A (0)	A	-	11 (C)	?	?	=	=



No, ya que los sucesores del nodo actual A no lo tienen como soporte (mejor vecino).



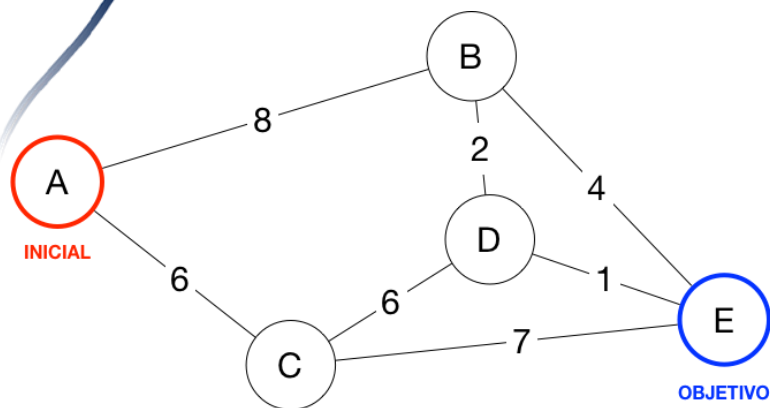
c(x,y)+h(y) en vecinos del nodo actual				
y=A	y=B	y=C	y=D	y=E
	12	11		

Estando en A

	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Nodo Actual (Coste)	Camino recorrido	Sig. nodo	Tabla heurística y soporte				
				h(A)	h(B)	h(C)	h(D)	h(E)
0	-	-	-	10 (Ø)	4 (Ø)	5 (Ø)	1 (Ø)	0 (Ø)
1	A (0)	A	C	11 (C)	?	?	=	=

Y ahora, tras la actualización heurística, con todos los costes actualizados, es cuando aplicamos la regla de movimiento de LRTA*.



c(x,y)+h(y) en vecinos del nodo actual				
y=A	y=B	y=C	y=D	y=E
	12	11		
17			7	7

Estando en A
Estando en C

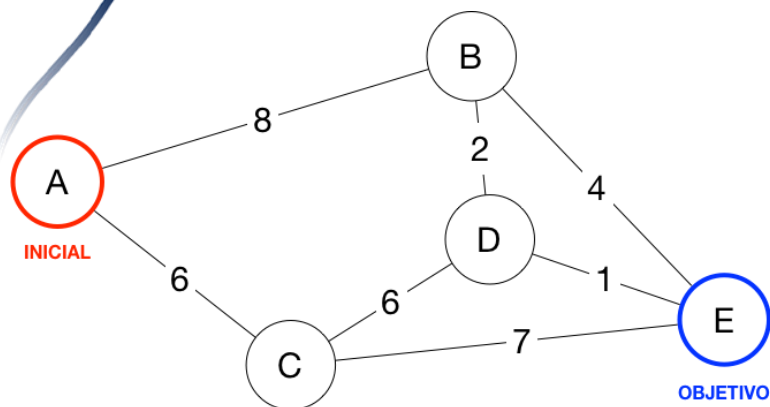
	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Nodo Actual (Coste)	Camino recorrido	Sig. nodo	Tabla heurística y soporte				
				h(A)	h(B)	h(C)	h(D)	h(E)
0	-	-	-	10 (Ø)	4 (Ø)	5 (Ø)	1 (Ø)	0 (Ø)
1	A (0)	A	C	11 (C)	=	=	=	=
2	C (6)	A - C	-	?	=	7 (D)	?	?

$c(C,D)+h(D)=6+1$



Hemos actualizado el nodo C. ¿Deberíamos actualizar a sus sucesores (A, D, E) por la propagación?



	A	B	C	D	E
$h(n)$	10	4	5	1	0

Estando en A
Estando en C
Estando en A (propagación)

$c(x,y)+h(y)$ en vecinos del nodo actual				
$y=A$	$y=B$	$y=C$	$y=D$	$y=E$
	12	11		
17			7	7
	12	13		

It.	Nodo Actual (Coste)	Camino recorrido	Sig. nodo	Tabla heurística y soporte				
				$h(A)$	$h(B)$	$h(C)$	$h(D)$	$h(E)$
0	-	-	-	10 ($\emptyset$)	4 ($\emptyset$)	5 ($\emptyset$)	1 ($\emptyset$)	0 ($\emptyset$)
1	A (0)	A	C	11 (C)	=	=	=	=
2	C (6)	A - C	-	12 (B)	=	7 (D)	=	=

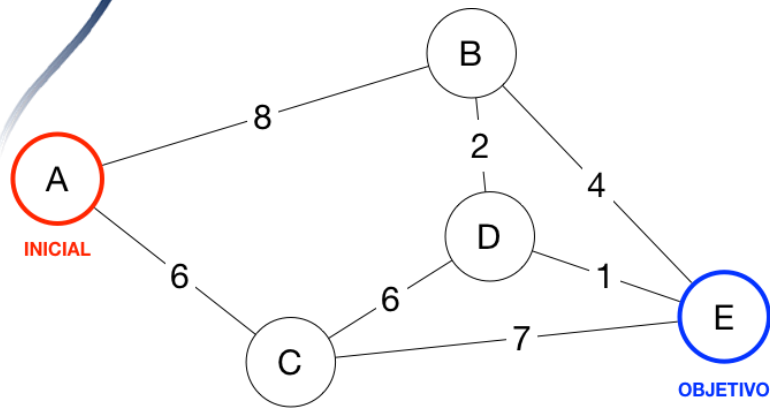
$$c(A,B)+h(B)=8+4$$

$$c(C,D)+h(D)=6+1$$



Podemos actualizar tan solo 1 de los 3 (si hay alguno válido) porque es LRTA*(2). A se puede actualizar porque se cumple la condición:

- El nodo que se actualizó (C) es el soporte de A.



	A	B	C	D	E
$h(n)$	10	4	5	1	0

Estando en A

Estando en C

Estando en A (propagación)

Estando en C (movimiento)

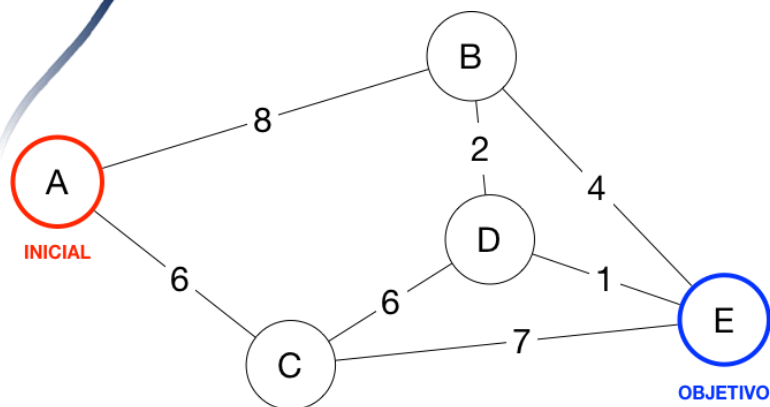
$c(x,y)+h(y)$ en vecinos del nodo actual				
$y=A$	$y=B$	$y=C$	$y=D$	$y=E$
	12	11		
17			7	7
	12	13		
18			7	7

It.	Nodo Actual (Coste)	Camino recorrido	Sig. nodo	Tabla heurística y soporte				
				$h(A)$	$h(B)$	$h(C)$	$h(D)$	$h(E)$
0	-	-	-	10 ($\emptyset$)	4 ($\emptyset$)	5 ($\emptyset$)	1 ($\emptyset$)	0 ($\emptyset$)
1	A (0)	A	C	11 (C)	=	=	=	=
2	C (6)	A - C	D	12 (B)	=	7 (D)	=	=

$$c(A,B)+h(B)=8+4$$

$$c(C,D)+h(D)=6+1$$

Ahora, para cumplir con la estrategia de movimiento, debemos recalcular el valor de coste de desplazamiento más heurística (tabla superior derecha), ya que la propagación afectó a uno de sus vecinos.



Estando en A
(propagación)
Estando en C
(movimiento)
Estando en D

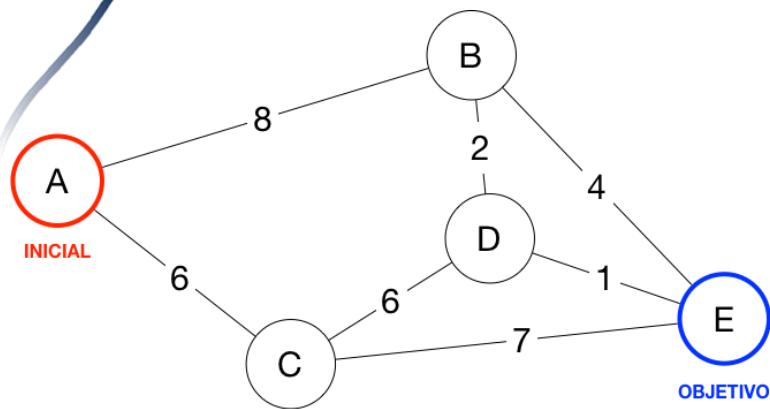
c(x,y)+h(y) en vecinos del nodo actual				
y=A	y=B	y=C	y=D	y=E
	12	13		
18			7	7
	6	13		1

	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Nodo Actual (Coste)	Camino recorrido	Sig. nodo	Tabla heurística y soporte				
				h(A)	h(B)	h(C)	h(D)	h(E)
0	-	-	-	10 (Ø)	4 (Ø)	5 (Ø)	1 (Ø)	0 (Ø)
1	A (0)	A	C	11 (C)	=	=	=	=
2	C (6)	A - C	D	12 (B)	=	7 (D)	=	=
3	D (12)	A - C - D	E	=	=	=	1 (E)	=

$c(D,E)+h(E)=1+0$

No se propaga nada porque no se actualizan valores heurísticos.



Estando en A
(propagación)
Estando en C
(movimiento)
Estando en D

c(x,y)+h(y) en vecinos del nodo actual				
y=A	y=B	y=C	y=D	y=E
	12	13		
18			7	7
	6	13		1

	A	B	C	D	E
h(n)	10	4	5	1	0

It.	Nodo Actual (Coste)	Camino recorrido	Sig. nodo	Tabla heurística y soporte				
				h(A)	h(B)	h(C)	h(D)	h(E)
0	-	-	-	10 (Ø)	4 (Ø)	5 (Ø)	1 (Ø)	0 (Ø)
1	A (0)	A	C	11 (C)	=	=	=	=
2	C (6)	A - C	D	12 (B)	=	7 (D)	=	=
3	D (12)	A - C - D	E	=	=	=	1 (E)	=
4	E (13)	A - C - D - E (Encontrado)		12 (B)	4 (Ø)	7 (D)	1 (E)	0 (Ø)



NODO
ACTUAL



NODO META



VECINO DEL
NODO
ACTUAL



MEJOR VECINO DEL NODO ACTUAL
(AL QUE NOS MOVEREMOS POR LA
ESTRATEGIA DE MOVIMIENTO)



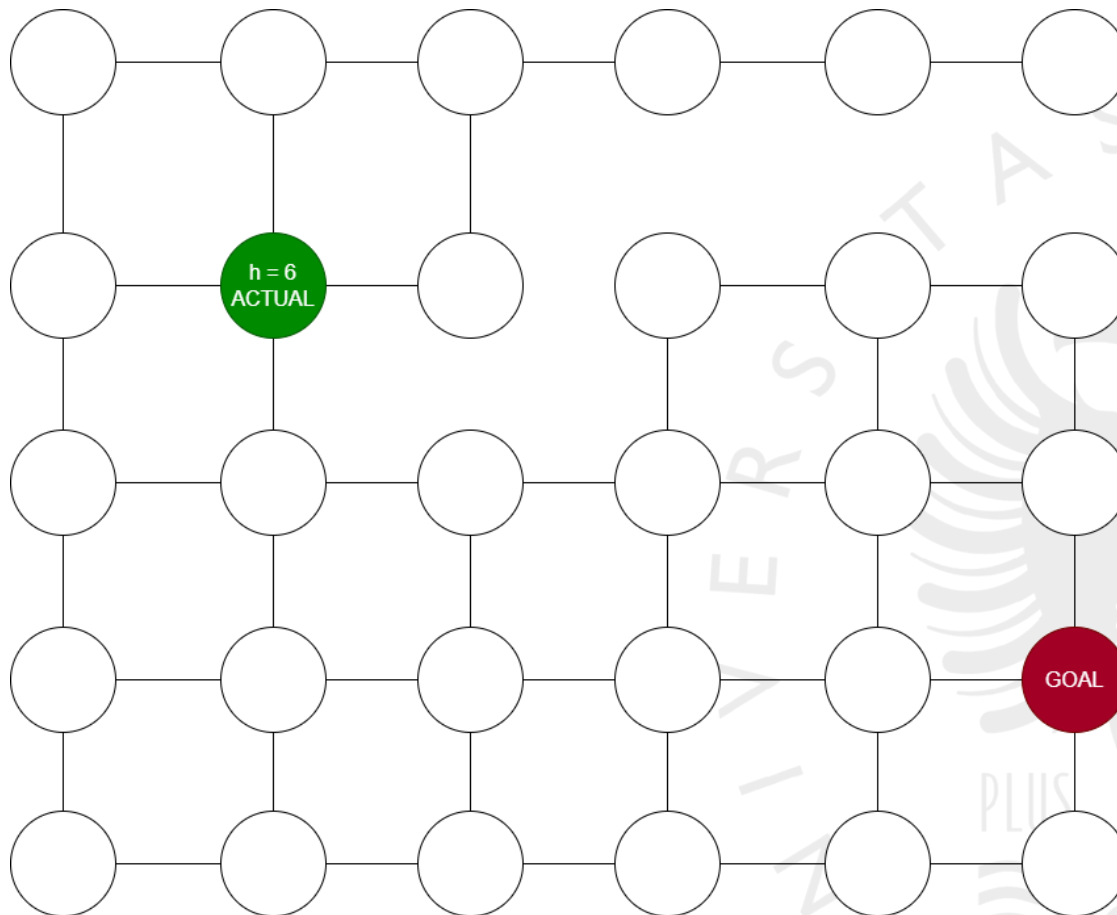
NODO X DE LA COLA
DE PROPAGACIÓN



VECINOS DEL NODO X
DE LA COLA DE
PROPAGACIÓN



VECINO DEL NODO X DE LA COLA DE PROPAGACIÓN QUE TIENE DE
SOPORTE AL NODO X (ES DECIR, SE PUEDE METER EN LA COLA SI
EL CONTADOR ES LO SUFICIENTEMENTE ALTO)





NODO
ACTUAL



NODO META



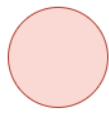
VECINO DEL
NODO
ACTUAL



MEJOR VECINO DEL NODO ACTUAL
(AL QUE NOS MOVEREMOS POR LA
ESTRATEGIA DE MOVIMIENTO)



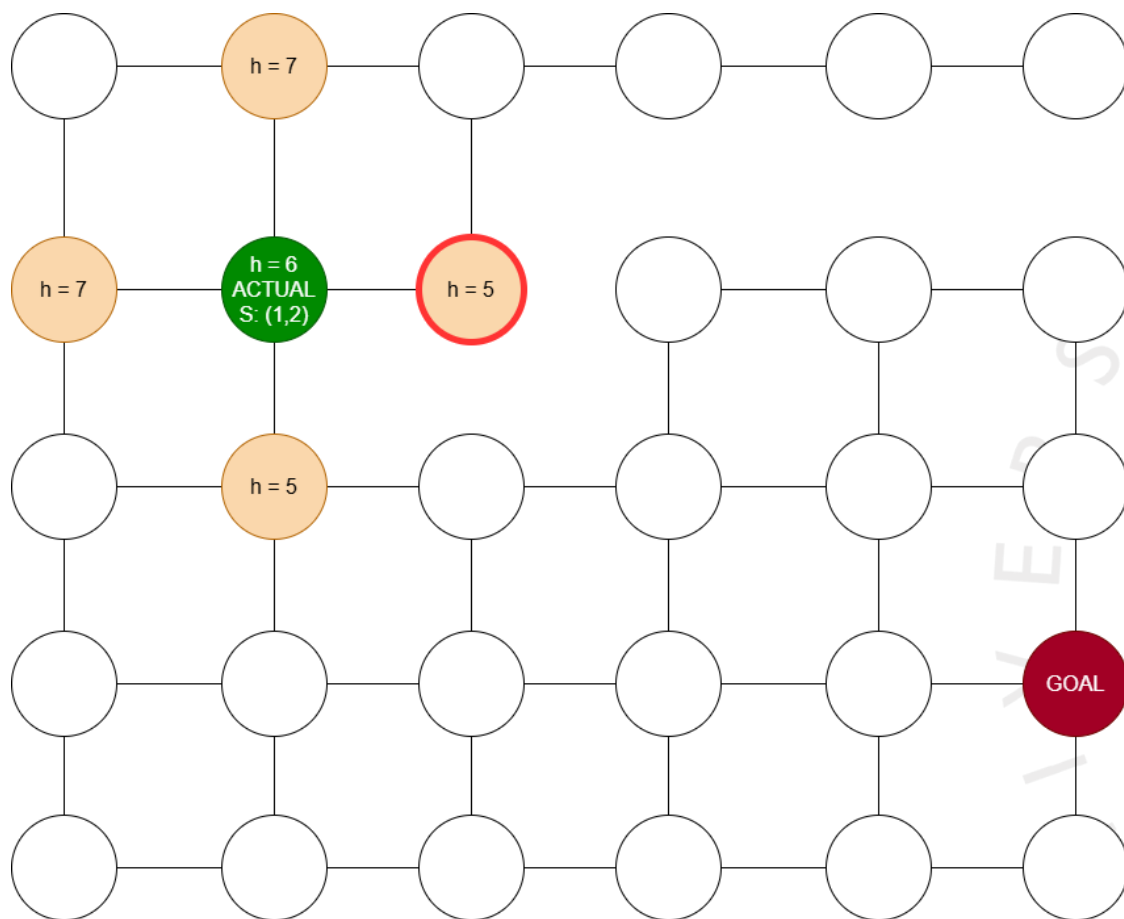
NODO X DE LA COLA
DE PROPAGACIÓN



VECINOS DEL NODO X
DE LA COLA DE
PROPAGACIÓN



VECINO DEL NODO X DE LA COLA DE PROPAGACIÓN QUE TIENE DE
SOPORTE AL NODO X (ES DECIR, SE PUEDE METER EN LA COLA SI
EL CONTADOR ES LO SUFICIENTEMENTE ALTO)



En este caso, la regla de
actualización de LRTA* no se
cumple porque

$$h(\text{ACTUAL}) \nless c(\text{ACTUAL}, \text{mejorVecino}) + h(\text{mejorVecino})$$

Por tanto, no es necesario
actualizar el valor heurístico de
ACTUAL, su soporte es su mejor
vecino (fila=1, columna=2) [que
aparece remarcado en rojo en la
figura], y nos movemos a dicho
vecino.



NODO
ACTUAL



NODO META



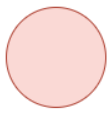
VECINO DEL
NODO
ACTUAL



MEJOR VECINO DEL NODO ACTUAL
(AL QUE NOS MOVEREMOS POR LA
ESTRATEGIA DE MOVIMIENTO)



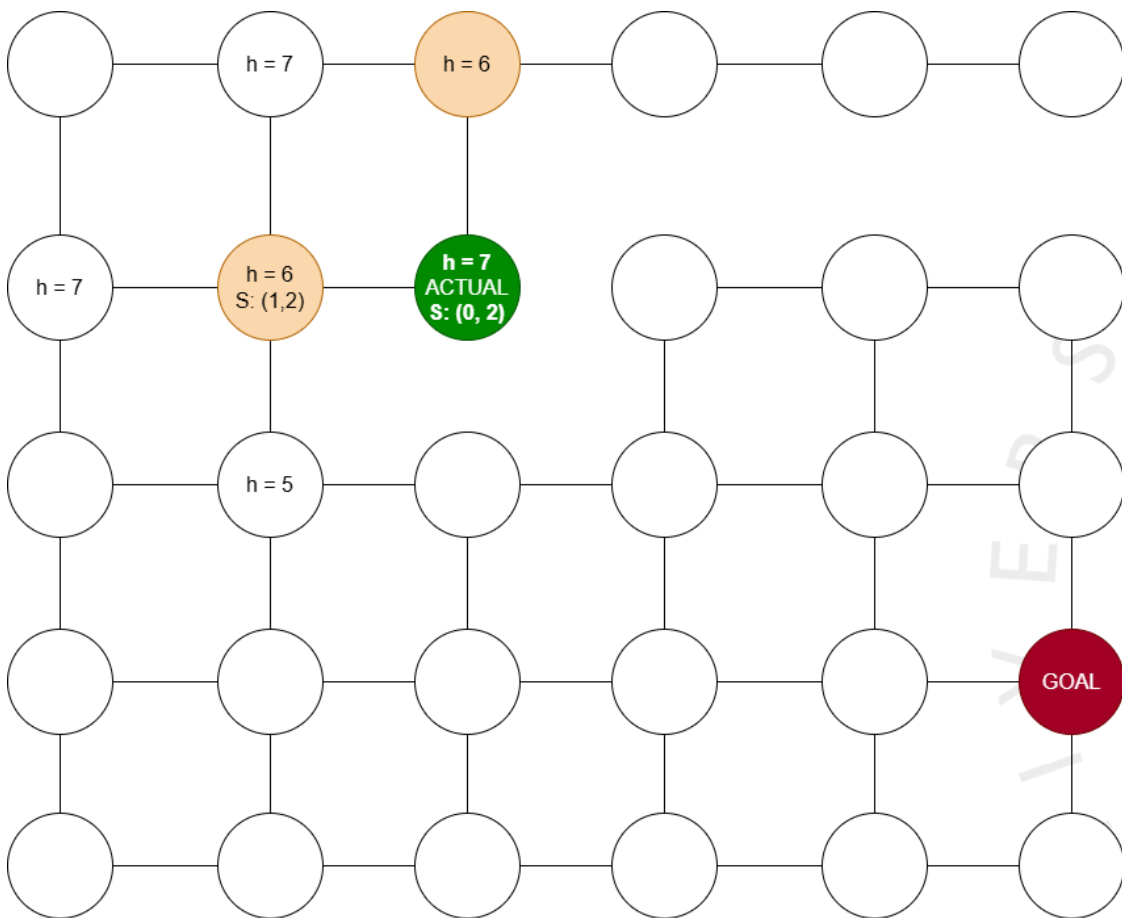
NODO X DE LA COLA
DE PROPAGACIÓN



VECINOS DEL NODO X DE LA COLA DE PROPAGACIÓN



VECINO DEL NODO X DE LA COLA DE PROPAGACIÓN QUE TIENE DE SOPORTE AL NODO X (ES DECIR, SE PUEDE METER EN LA COLA SI EL CONTADOR ES LO SUFICIENTEMENTE ALTO)



En este caso, la regla de actualización de LRTA* sí se cumple porque

$$h(\text{ACTUAL}) < c(\text{ACTUAL}, \text{mejorVecino}) + h(\text{mejorVecino})$$

$$5 < 1 + 6 = 7$$

Por tanto, actualizamos el valor heurístico de ACTUAL, pasando de 5 a 7, su soporte es su mejor vecino (0,2) y, como hemos realizado la actualización, aumentamos el número de actualizaciones realizadas en la tabla heurística, y revisamos si debemos propagar.



NODO
ACTUAL



NODO META



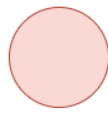
VECINO DEL
NODO
ACTUAL



MEJOR VECINO DEL NODO ACTUAL
(AL QUE NOS MOVEREMOS POR LA
ESTRATEGIA DE MOVIMIENTO)



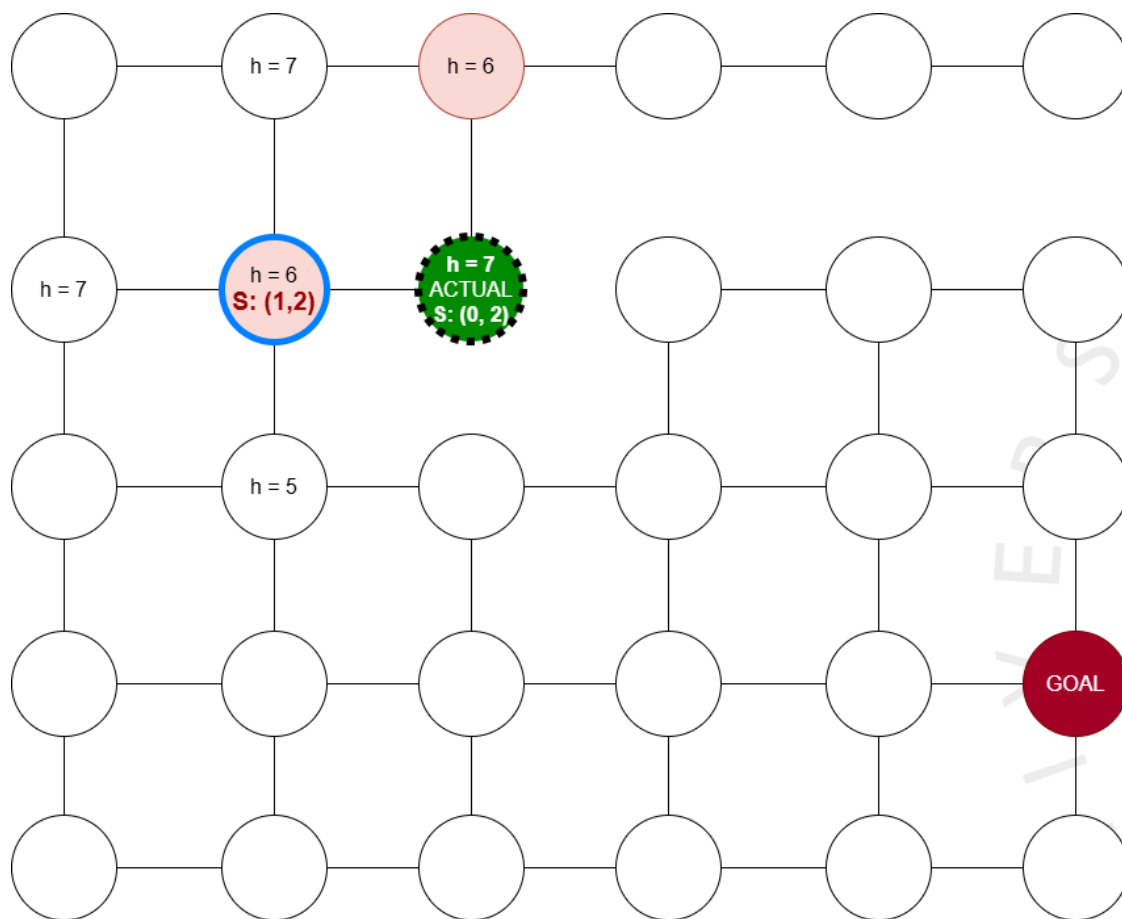
NODO X DE LA COLA
DE PROPAGACIÓN



VECINOS DEL NODO X
DE LA COLA DE
PROPAGACIÓN



VECINO DEL NODO X DE LA COLA DE PROPAGACIÓN QUE TIENE DE
SOPORTE AL NODO X (ES DECIR, SE PUEDE METER EN LA COLA SI
EL CONTADOR ES LO SUFICIENTEMENTE ALTO)



Entramos en la propagación:

Cola = $\langle (1,2) \rangle$

contador = 2

Sacamos el nodo (1,2).

Para propagar desde un nodo X (en este caso, el (1,2)), tiene que ocurrir:

1. Que hayamos actualizado la heurística de X ($5 \rightarrow 7$)
2. Soporte(Sucesor) == X
3. contador > 0

En ese caso, dicho nodo Sucesor se mete en la cola. Esto se cumple para (1,1) [resaltado con un borde azul]. Por tanto:

Cola = $\langle (1,1) \rangle$

contador = 1



NODO
ACTUAL



NODO META



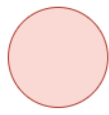
VECINO DEL
NODO
ACTUAL



MEJOR VECINO DEL NODO ACTUAL
(AL QUE NOS MOVEREMOS POR LA
ESTRATEGIA DE MOVIMIENTO)



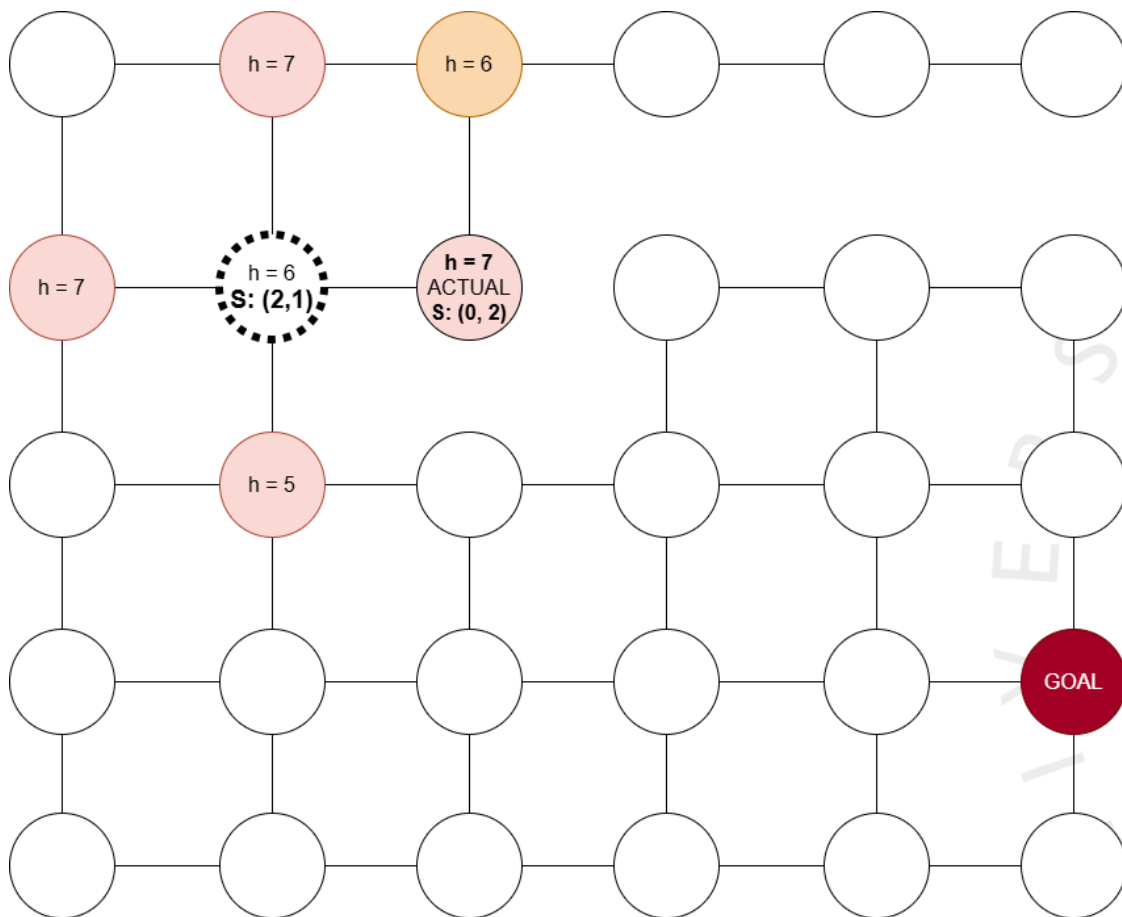
NODO X DE LA COLA
DE PROPAGACIÓN



VECINOS DEL NODO X
DE LA COLA DE
PROPAGACIÓN



VECINO DEL NODO X DE LA COLA DE PROPAGACIÓN QUE TIENE DE
SOPORTE AL NODO X (ES DECIR, SE PUEDE METER EN LA COLA SI
EL CONTADOR ES LO SUFICIENTEMENTE ALTO)



Seguimos con la propagación:
Cola = $\langle (1,1) \rangle$
contador = 1

Sacamos el nodo (1,1).

En este caso, no se actualiza la
heurística, pero sí que cambia
el soporte, pasando de (1,2) a
(2,1).

Puesto que no se ha
actualizado la heurística, no
hay posibilidad de añadir más
elementos a la cola.

Como la cola ya está vacía,
finalizamos la propagación.



NODO
ACTUAL



NODO META



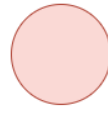
VECINO DEL
NODO
ACTUAL



MEJOR VECINO DEL NODO ACTUAL
(AL QUE NOS MOVEREMOS POR LA
ESTRATEGIA DE MOVIMIENTO)



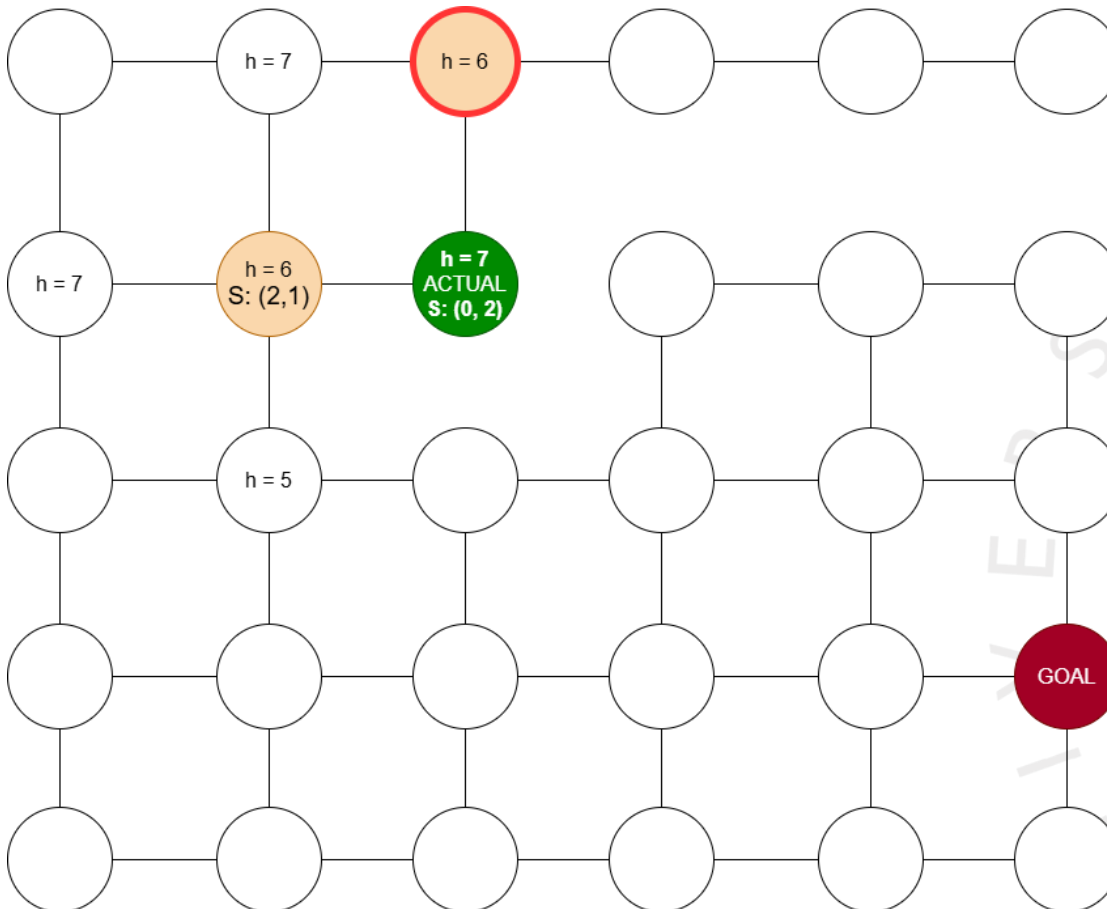
NODO X DE LA COLA
DE PROPAGACIÓN



VECINOS DEL NODO X
DE LA COLA DE
PROPAGACIÓN



VECINO DEL NODO X DE LA COLA DE PROPAGACIÓN QUE TIENE DE
SOPORTE AL NODO X (ES DECIR, SE PUEDE METER EN LA COLA SI
EL CONTADOR ES LO SUFICIENTEMENTE ALTO)



Tras finalizar la propagación,
determinamos a dónde debemos
movernos. Según LRTA*
debemos movernos a nuestro
mejor vecino.

En este caso, nos podríamos
mover tanto a (0,2) como a (1,1)
pero, para desempatar, seguimos
el orden de preferencia
establecido para la generación
de sucesores (DERECHA,
ARRIBA, IZQUIERDA, ABAJO).

Por tanto, nos movemos a (0,2).



NODO
ACTUAL



NODO META



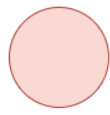
VECINO DEL
NODO
ACTUAL



MEJOR VECINO DEL NODO ACTUAL
(AL QUE NOS MOVEREMOS POR LA
ESTRATEGIA DE MOVIMIENTO)



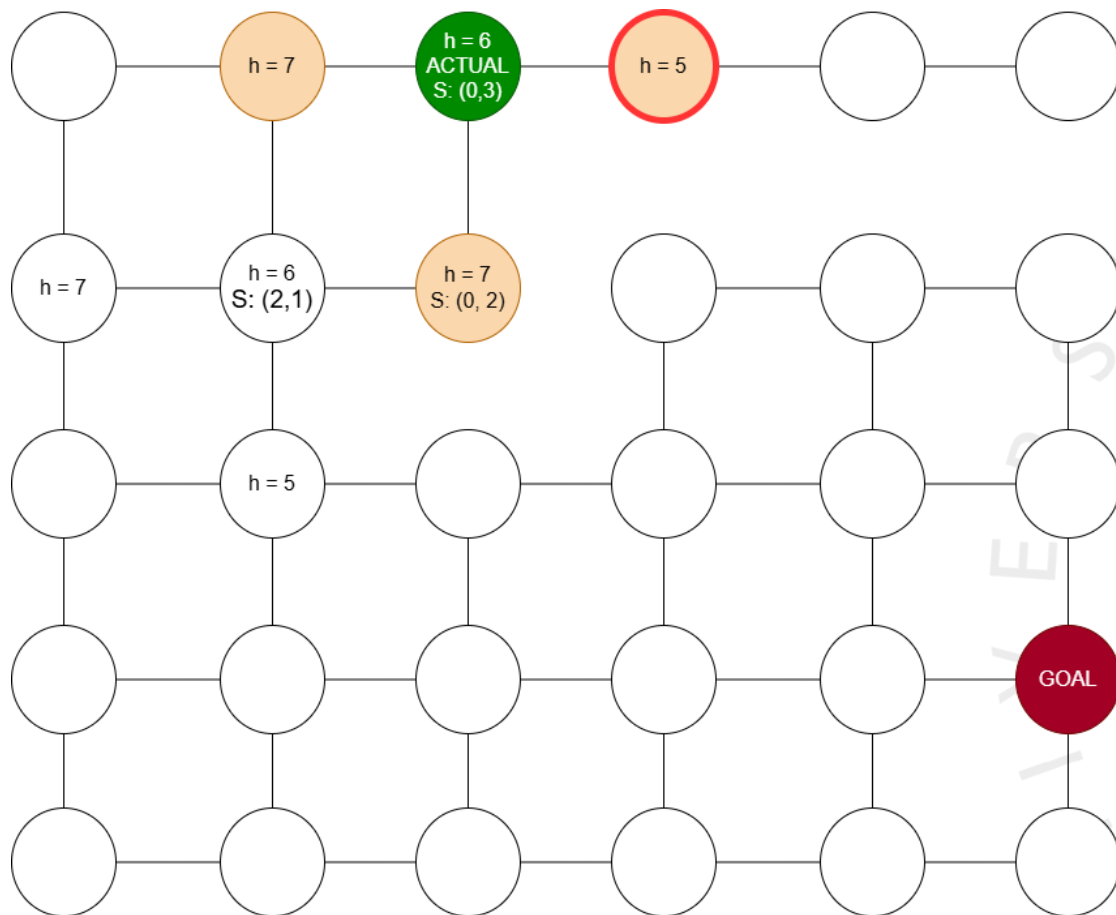
NODO X DE LA COLA
DE PROPAGACIÓN



VECINOS DEL NODO X
DE LA COLA DE
PROPAGACIÓN



VECINO DEL NODO X DE LA COLA DE PROPAGACIÓN QUE TIENE DE
SOPORTE AL NODO X (ES DECIR, SE PUEDE METER EN LA COLA SI
EL CONTADOR ES LO SUFICIENTEMENTE ALTO)



En este caso, la regla de
actualización de LRTA* no se
cumple porque

$$h(Actual) \nless c(Actual, mejorVecino) + h(mejorVecino)$$

$$6 \nless 1 + 5$$

Por tanto, no es necesario
actualizar el valor heurístico de
ACTUAL, su soporte es su
mejor vecino (0,3), y nos
movemos al mejor vecino.



NODO
ACTUAL



NODO META



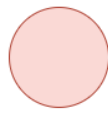
VECINO DEL
NODO
ACTUAL



MEJOR VECINO DEL NODO ACTUAL
(AL QUE NOS MOVEREMOS POR LA
ESTRATEGIA DE MOVIMIENTO)



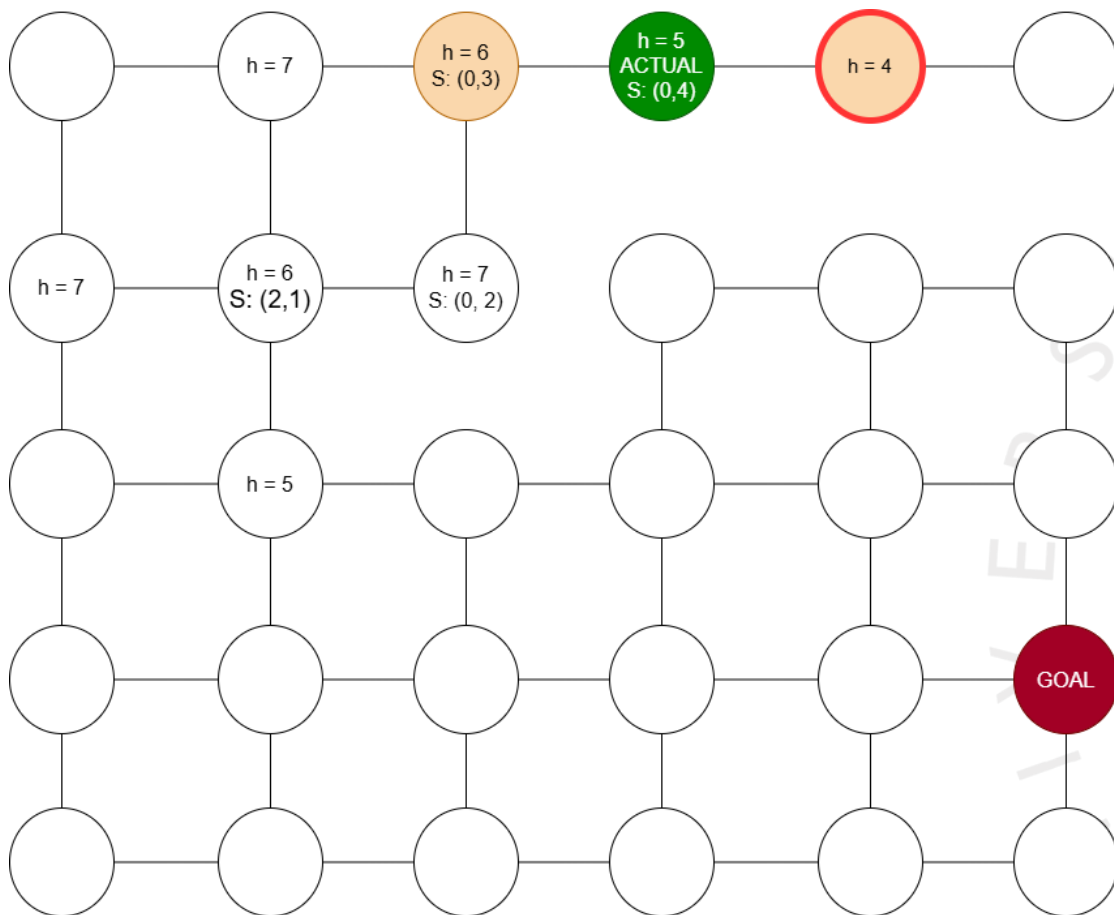
NODO X DE LA COLA
DE PROPAGACIÓN



VECINOS DEL NODO X
DE LA COLA DE
PROPAGACIÓN



VECINO DEL NODO X DE LA COLA DE PROPAGACIÓN QUE TIENE DE
SOPORTE AL NODO X (ES DECIR, SE PUEDE METER EN LA COLA SI
EL CONTADOR ES LO SUFICIENTEMENTE ALTO)



En este caso, la regla de
actualización de LRTA* no se
cumple porque

$$h(\text{ACTUAL}) \nless c(\text{ACTUAL}, \text{mejorVecino}) + h(\text{mejorVecino})$$

$$5 \nless 1 + 4$$

Por tanto, no es necesario
actualizar el valor heurístico de
ACTUAL, su soporte es su
mejor vecino (0,4), y nos
movemos al mejor vecino.



NODO
ACTUAL



NODO META



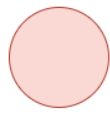
VECINO DEL
NODO
ACTUAL



MEJOR VECINO DEL NODO ACTUAL
(AL QUE NOS MOVEREMOS POR LA
ESTRATEGIA DE MOVIMIENTO)



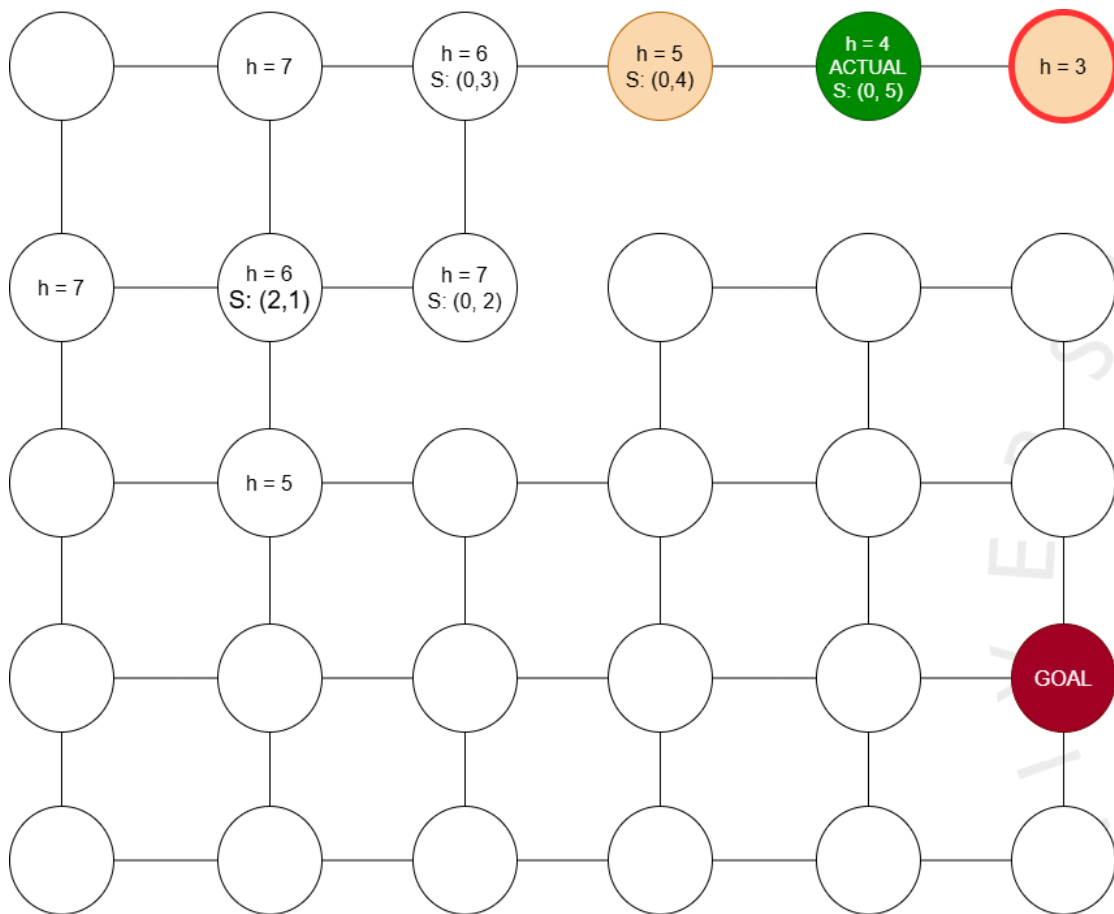
NODO X DE LA COLA
DE PROPAGACIÓN



VECINOS DEL NODO X
DE LA COLA DE
PROPAGACIÓN



VECINO DEL NODO X DE LA COLA DE PROPAGACIÓN QUE TIENE DE
SOPORTE AL NODO X (ES DECIR, SE PUEDE METER EN LA COLA SI
EL CONTADOR ES LO SUFICIENTEMENTE ALTO)



En este caso, la regla de
actualización de LRTA* no se
cumple porque

$$h(Actual) \nless c(Actual, mejorVecino) + h(mejorVecino)$$

$$4 \nless 1 + 3$$

Por tanto, no es necesario
actualizar el valor heurístico de
ACTUAL, su soporte es su
mejor vecino (0,5), y nos
movemos al mejor vecino.



NODO
ACTUAL



NODO META



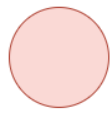
VECINO DEL
NODO
ACTUAL



MEJOR VECINO DEL NODO ACTUAL
(AL QUE NOS MOVEREMOS POR LA
ESTRATEGIA DE MOVIMIENTO)



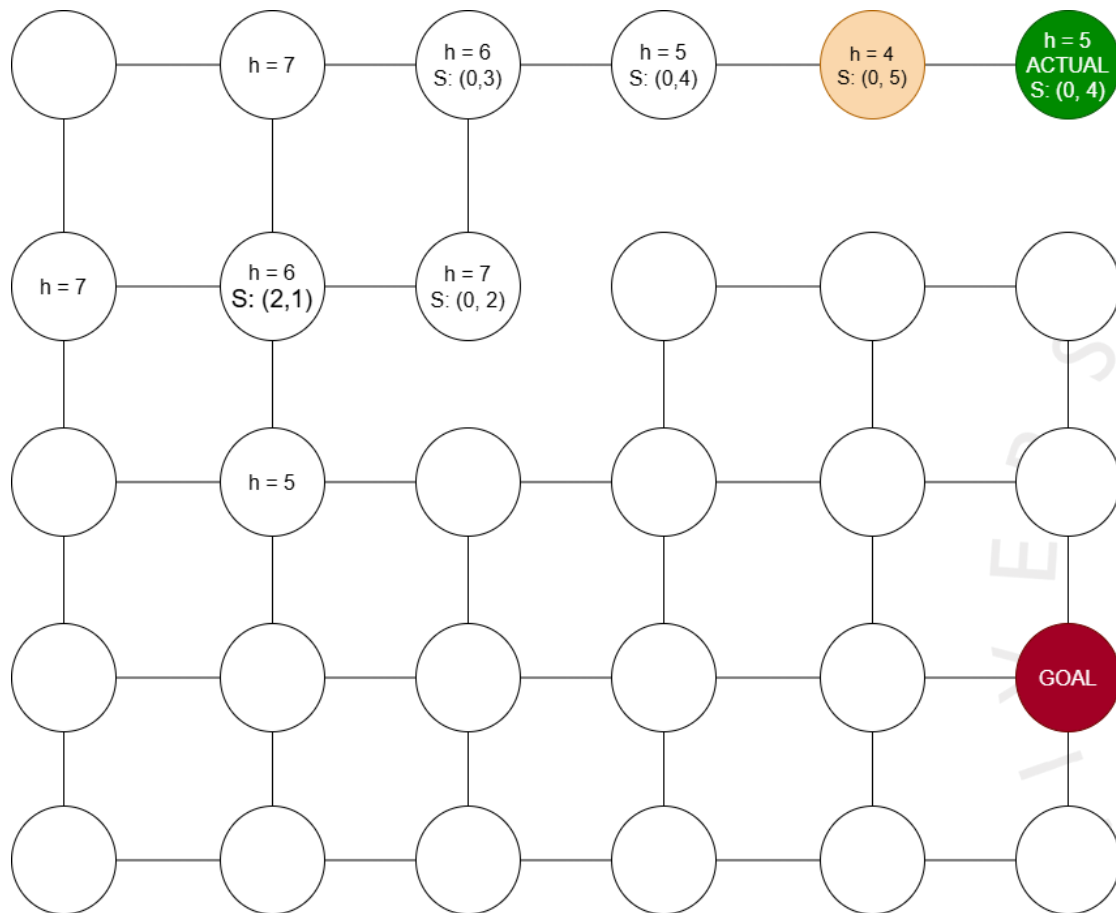
NODO X DE LA COLA
DE PROPAGACIÓN



VECINOS DEL NODO X
DE LA COLA DE
PROPAGACIÓN



VECINO DEL NODO X DE LA COLA DE PROPAGACIÓN QUE TIENE DE
SOPORTE AL NODO X (ES DECIR, SE PUEDE METER EN LA COLA SI
EL CONTADOR ES LO SUFICIENTEMENTE ALTO)



Ahora, la regla de actualización
de LRTA* sí se cumple porque

$$h(\text{ACTUAL}) < c(\text{ACTUAL}, \text{mejorVecino}) + h(\text{mejorVecino})$$

$$3 < 1 + 4$$

Por tanto, es necesario
actualizar el valor heurístico de
ACTUAL (pasando de 3 a 5),
su soporte es su mejor vecino
(0,4), y debemos estudiar si
tenemos que propagar.



NODO
ACTUAL



NODO META



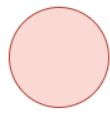
VECINO DEL
NODO
ACTUAL



MEJOR VECINO DEL NODO ACTUAL
(AL QUE NOS MOVEREMOS POR LA
ESTRATEGIA DE MOVIMIENTO)



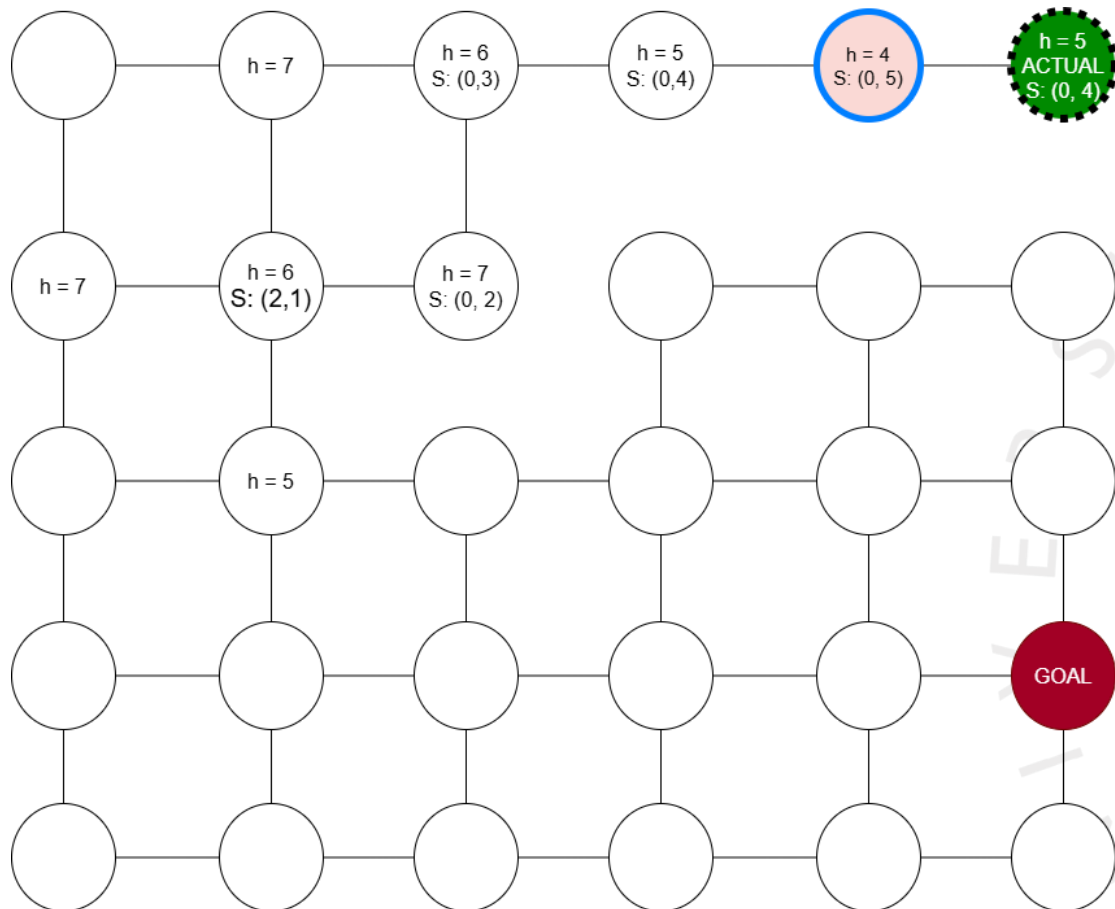
NODO X DE LA COLA
DE PROPAGACIÓN



VECINOS DEL NODO X
DE LA COLA DE
PROPAGACIÓN



VECINO DEL NODO X DE LA COLA DE PROPAGACIÓN QUE TIENE DE
SOPORTE AL NODO X (ES DECIR, SE PUEDE METER EN LA COLA SI
EL CONTADOR ES LO SUFICIENTEMENTE ALTO)



Entramos en la propagación:

Cola = $\langle (0,5) \rangle$

contador = 2

Sacamos el nodo $(0,5)$.

Para propagar desde un nodo X (en este caso, el $(0,5)$), tiene que ocurrir:

1. Que hayamos actualizado la heurística de X ($3 \rightarrow 5$)
2. $\text{Soporte}(\text{Sucesor}) == X$
3. contador > 0

En ese caso, dicho nodo Sucesor se mete en la cola. Esto se cumple para $(0,4)$ [resaltado con un borde azul].

Por tanto:

Cola = $\langle (0,4) \rangle$

contador = 1



NODO
ACTUAL



NODO META



VECINO DEL
NODO
ACTUAL



MEJOR VECINO DEL NODO ACTUAL
(AL QUE NOS MOVEREMOS POR LA
ESTRATEGIA DE MOVIMIENTO)



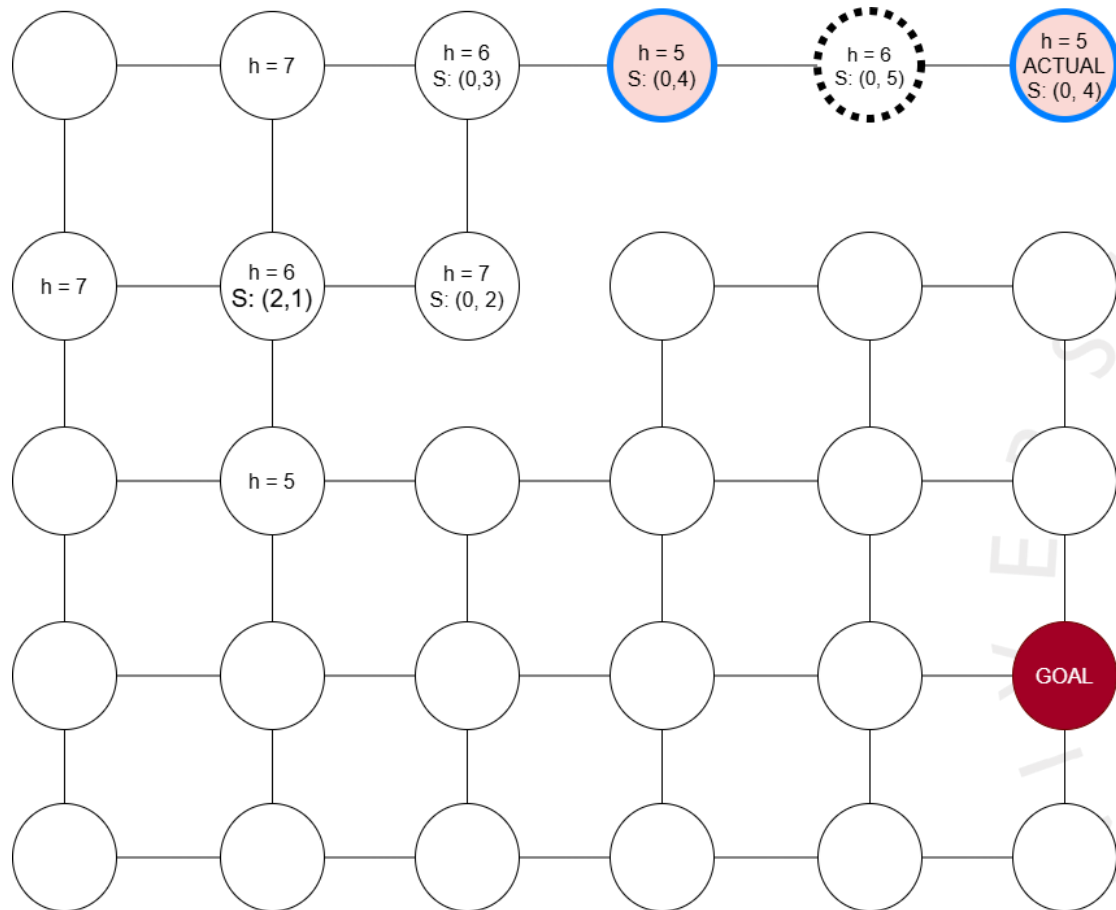
NODO X DE LA COLA DE PROPAGACIÓN



VECINOS DEL NODO X DE LA COLA DE PROPAGACIÓN



VECINO DEL NODO X DE LA COLA DE PROPAGACIÓN QUE TIENE DE SOPORTE AL NODO X (ES DECIR, SE PUEDE METER EN LA COLA SI EL CONTADOR ES LO SUFICIENTEMENTE ALTO)



Seguimos en la propagación:

Cola = $\langle (0,4) \rangle$

contador = 1

Sacamos el nodo (0,4).

Para propagar desde un nodo X (en este caso, el (0,4)), tiene que ocurrir:

1. Que hayamos actualizado la heurística de X ($4 \rightarrow 6$)
2. $\text{Soporte}(\text{Sucesor}) == X$
3. $\text{contador} > 0$

En ese caso, dicho nodo Sucesor se mete a la cola. Esto se cumple para $(0,3)$ y $(0,5)$ [resaltados con un borde azul]. Como contador es 1, solo se puede meter uno de los dos.

Priorizamos según el orden de expansión y hacemos:

Cola = $\langle (0,5) \rangle$

```
contador = 0
```



NODO
ACTUAL



NODO META



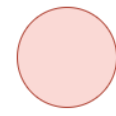
VECINO DEL
NODO
ACTUAL



MEJOR VECINO DEL NODO ACTUAL
(AL QUE NOS MOVEREMOS POR LA
ESTRATEGIA DE MOVIMIENTO)



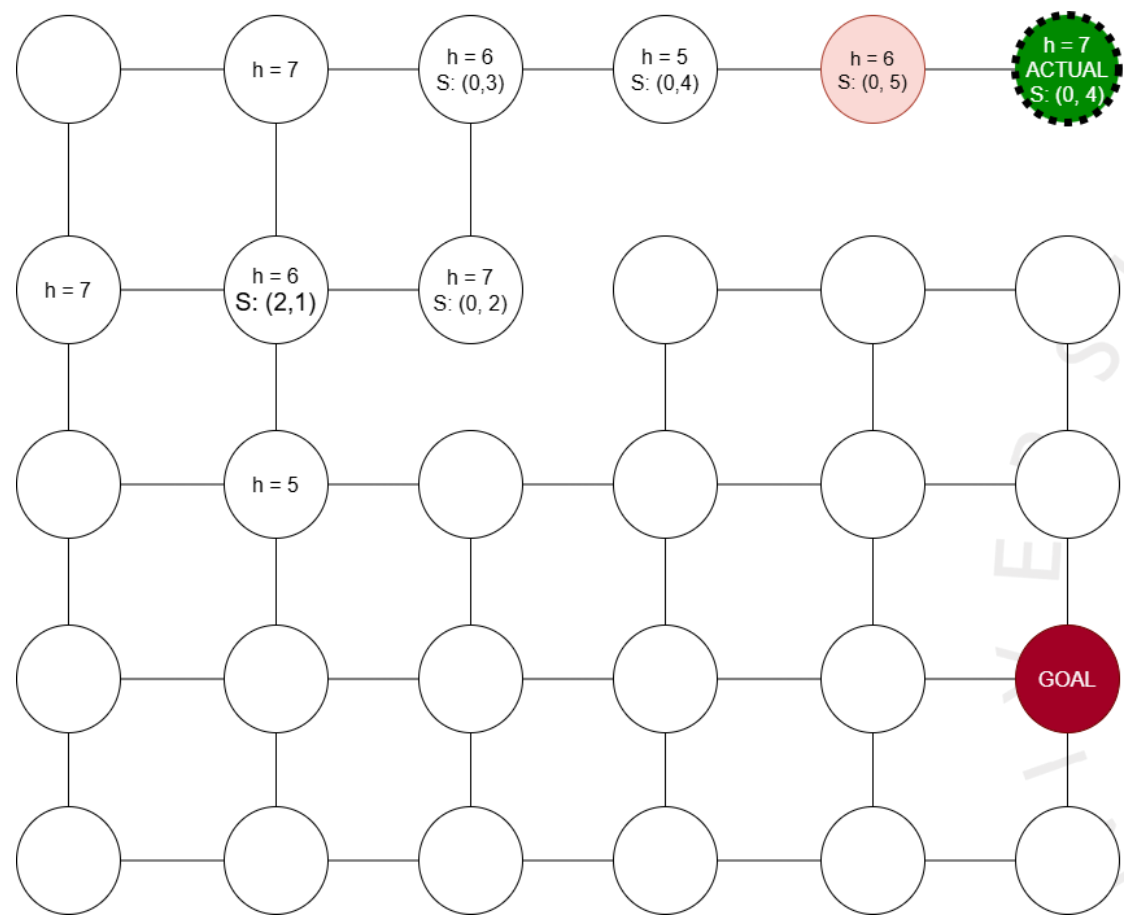
NODO X DE LA COLA
DE PROPAGACIÓN



VECINOS DEL NODO X
DE LA COLA DE
PROPAGACIÓN



VECINO DEL NODO X DE LA COLA DE PROPAGACIÓN QUE TIENE DE
SOPORTE AL NODO X (ES DECIR, SE PUEDE METER EN LA COLA SI
EL CONTADOR ES LO SUFICIENTEMENTE ALTO)



Seguimos con la propagación:

Cola = $\langle (0,5) \rangle$

contador = 0

Sacamos el nodo (0,5). Como contador ya es 0, podemos actualizar el valor heurístico del nodo que hemos sacado de la cola (5 $\rightarrow$ 7), pero no habrá más propagaciones de valores heurísticos.

Termina la propagación.



NODO
ACTUAL



NODO META



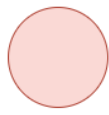
VECINO DEL
NODO
ACTUAL



MEJOR VECINO DEL NODO ACTUAL
(AL QUE NOS MOVEREMOS POR LA
ESTRATEGIA DE MOVIMIENTO)



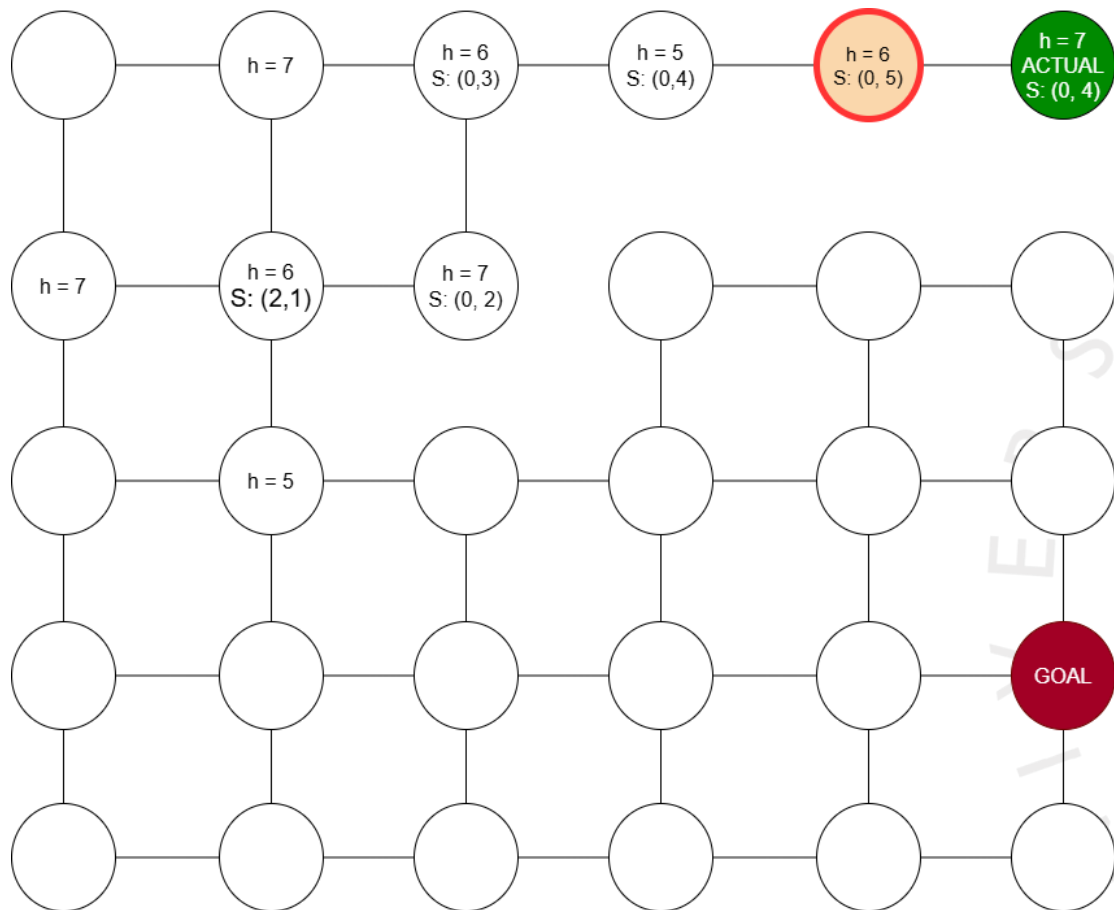
NODO X DE LA COLA
DE PROPAGACIÓN



VECINOS DEL NODO X
DE LA COLA DE
PROPAGACIÓN



VECINO DEL NODO X DE LA COLA DE PROPAGACIÓN QUE TIENE DE
SOPORTE AL NODO X (ES DECIR, SE PUEDE METER EN LA COLA SI
EL CONTADOR ES LO SUFICIENTEMENTE ALTO)



Tras finalizar la propagación,
determinamos a dónde debemos
movernos. Según LRTA* debemos
movernos a nuestro mejor vecino.

En este caso, nos podemos mover
tan solo a (0,4).

Y el proceso continúa como se ha
explicado hasta ahora...